A project report on

MULTI-AGENT DEBATE FRAMEWORK USING EVIDENCE DISAGREEMENT TRIGGERING AND CONFLICT-TYPE CLASSIFICATION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

M. REDDY SASI KIRAN (23BAI1188)
SUHAS KOHEDA (23BAI1148)
C R SUHAS (23BAI1187)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI

October, 2026

A project report on

MULTI-AGENT DEBATE FRAMEWORK USING EVIDENCE DISAGREEMENT TRIGGERING AND CONFLICT-TYPE CLASSIFICATION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

M. REDDY SASI KIRAN (23BAI1188)
SUHAS KOHEDA (23BAI1148)
C R SUHAS (23BAI1187)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI

October, 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I hereby declare that the thesis entitled “Multi-Agent Debate Framework Using Evidence Disagreement Triggering and Conflict-Type Classification” submitted by C R SUHAS (23BAI1187), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr M. Marimuthu .

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date:

Signature of the Candidate

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled **"Multi-Agent Debate Framework Using Evidence Disagreement Triggering and Conflict-Type Classification"** is prepared and submitted by C R SUHAS (23BAI1187) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Artificial Intelligence and Machine Learning** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. M. Marimuthu

Date:

Signature of the Examiner
Name:
Date:

Signature of the Examiner:
Name:
Date:

Approved by the Head of Department,
(Computer Science and Engineering, Artificial Intelligence and Machine Learning)

Name: Dr Tulasi Prasad Sariki
Date:

ABSTRACT

Large language models still hallucinate, and they often get dates and software versions wrong, which makes them hard to trust on factual questions. Multi-agent debate is one way to improve reliability, but existing systems debate every query, even when the agents already agree. That wastes compute and slows responses down without making the answer any better.

This report presents Evidence-MAD, a framework that debates only when there is something to debate. An evidence gate runs pairwise Natural Language Inference (NLI) checks across the agents' answers. When they agree, the answer is returned directly. When they disagree, a classifier labels the conflict as FACTUAL, TEMPORAL, VERSION, CONTEXTUAL, or SOURCE, and the proposer, critic, and reviser agents follow a debate strategy designed for that specific type of conflict.

Evidence comes from four agents: a web search agent (DuckDuckGo), a reference agent (Wikipedia), a scholarly agent (Crossref), and an AI answer agent built on Google Gemini with Search grounding. Their output goes through a content-word relevance filter and LLM-based claim normalization before anything is compared. Three query-aware NLI guards also handle failure patterns that generic NLI models tend to miss: entity substitution, explicit challenges to a claim, and target-resolution mismatches.

We evaluated Evidence-MAD against five configurations: Single LLM, Standard MAD, No Gate, No Typed Strategy, and No Relevance Filter. On a controlled dataset, the evidence gate removed unnecessary debates without lowering accuracy, and the typed strategies improved how well temporal, version, and contextual conflicts were resolved. The system is deployed as a FastAPI service with a browser frontend, supports both offline fixture mode and live mode, and can optionally run on Vercel's serverless infrastructure.

Keywords: multi-agent debate, evidence gating, natural language inference, conflict-type classification, hallucination mitigation, large language models, information retrieval

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr M. Marimuthu, Assistant Professor (Sr.), School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with his is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Agentic AI.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing knowledge and for helping in every aspect of this academic journey.

In jubilant state, I express ingeniously my whole-hearted thanks to Dr Tulasi Prasad Sariki, Head of the Department, B.Tech. Artificial Intelligence and Machine Learning and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date:

C R SUHAS

CONTENTS

DECLARATION
CERTIFICATE
ABSTRACT.....i
ACKNOWLEDGMENT.....iii
LIST OF FIGURES.....vi
LIST OF TABLES.....vii
LIST OF ACRONYMS.....viii
CHAPTER 1 INTRODUCTION
1.1 Overview and Motivation.....1
1.2 Problem Statement.....2
1.3 Objectives.....2
1.4 Scope of the Project.....3
1.5 Organization of the Report.....3
CHAPTER 2 LITERATURE REVIEW
2.1 Introduction.....4
2.2 Multi-Agent Systems and Debate.....4
2.3 Natural Language Inference.....5
2.4 Evidence Retrieval Approaches.....6
2.5 Conflict Detection and Classification.....6
2.6 Research Gap.....7
CHAPTER 3 SYSTEM DESIGN AND ARCHITECTURE
3.1 High-Level Architecture.....8
3.2 Two-Mode Operation.....9
3.3 Component Dependency Map.....10
3.4 Configuration and Environment.....10
CHAPTER 4 AGENTS LAYER
4.1 Agent Registry and Base Class.....12
4.2 Query Router.....12
4.3 Web Agent.....14
4.4 Reference Agent.....14
4.5 Scholarly Agent.....15
4.6 AI Answer Agent.....15
CHAPTER 5 EVIDENCE PIPELINE
5.1 Evidence Flow Overview.....16
5.2 Relevance Filter.....16
5.3 LLM Normalization.....17
5.4 Evidence Aggregator.....17
CHAPTER 6 NLI AND CONFLICT DETECTION
6.1 NLI Detector Classes.....20
6.2 Query-Aware Guard Chain.....21
6.3 Entity Substitution Guard.....22

6.4 Explicit Challenge Guard....23
6.5 Target Resolution Guard....23
6.6 Transformer-Based NLI....24
CHAPTER 7 CONFLICT CLASSIFICATION
7.1 Conflict Classifier....25
7.2 Five-Type Taxonomy....25
7.3 Pattern-Based Fallback....26
CHAPTER 8 DEBATE LIFECYCLE
8.1 Debate Manager....27
8.2 Proposer....27
8.3 Critic....28
8.4 Reviser and Early Stopping....28
8.5 Judge....30
CHAPTER 9 API, FRONTEND AND DEPLOYMENT
9.1 FastAPI Application....31
9.2 Frontend....33
9.3 Deployment....34
CHAPTER 10 EVALUATION AND RESULTS
10.1 Evaluation Framework....35
10.2 Experimental Results....35
10.3 Ablation Study....38
CHAPTER 11 CONCLUSION AND FUTURE WORK....40
REFERENCES....42
APPENDICES....46

LIST OF FIGURES

Figure 1.1 Overview of the Evidence-MAD Pipeline....1
Figure 1.2 Evidence Gate Decision Flowchart....2
Figure 3.1 Evidence-Driven Multi-Agent Architecture....8
Figure 3.2 Fixture vs. Live Pipeline Modes....9
Figure 3.3 Component Dependency Map....10
Figure 4.1 QueryRouter Parallel Agent Workflow....13
Figure 4.2 Web Agent Search Workflow....14
Figure 4.3 AI Answer Agent Flowchart....15
Figure 5.1 Evidence Flow: Retrieval to Normalization....16
Figure 5.2 Content-Word Overlap Relevance Filter....17
Figure 5.3 Evidence Aggregation Decision Flowchart....18
Figure 6.1 NLI Detector Class Hierarchy Diagram....21
Figure 6.2 Query-Aware Guard Chain Flowchart....22
Figure 7.1 Conflict Types: Evidence-MAD Taxonomy....25
Figure 8.1 Debate Lifecycle Infographic Flowchart....27
Figure 8.2 Early Stop Debate Flowchart....29
Figure 9.1 Evidence Verification Pipeline Diagram....32
Figure 9.2 Screenshot of the Frontend....33
Figure 10.1 Evidence-MAD Evaluation Framework....35
Figure 10.2 Accuracy Comparison Across Datasets....37
Figure 10.3 Evidence-MAD System Architecture....37
Figure 10.4 Evidence-MAD System Pipeline....38
Figure 10.5 Hybrid Inference Pipeline Architecture....38
Figure 10.6 Conflict-Aware Multi-Agent Debate Architecture....39

LIST OF TABLES

Table 2.1 Summary of Related Work in Multi-Agent Debate.....5
Table 2.2 Comparison of NLI Models.....6
Table 3.1 Configuration Parameters and Defaults.....11
Table 4.1 Search Agent Summary.....12
Table 5.1 EvidenceStatus Values and Meanings.....19
Table 6.1 NLI Detector Class Comparison.....20
Table 7.1 Conflict Type Definitions and Examples.....26
Table 8.1 Debate Stop Reason Summary.....29
Table 9.1 API Response Schema Fields.....31
Table 10.1 Quantitative Evaluation Results Across Systems.....36
Table 10.2 Ablation Study Results.....39
Table 10.3 LLM Call Count by Role.....39

LIST OF ACRONYMS

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DDG	DuckDuckGo
DOI	Digital Object Identifier
F1	Harmonic Mean of Precision and Recall
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MAD	Multi-Agent Debate
NLI	Natural Language Inference
NLP	Natural Language Processing
RAG	Retrieval Augmented Generation
REST	Representational State Transfer
RoBERTa	Robustly Optimized BERT Pretraining Approach
SNLI	Stanford Natural Language Inference
URL	Uniform Resource Locator
VIT	Vellore Institute of Technology
QA	Question Answering
ASGI	Asynchronous Server Gateway Interface
CLI	Command Line Interface
SDK	Software Development Kit
TP	True Positive
FP	False Positive
TN	True Negative
FN	False Negative

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW AND MOTIVATION

LLMs like GPT-4, Gemini, and Llama do summarization, translation, code generation, and open-domain question answering well (Brown et al., 2020; Touvron et al., 2023), but they still hallucinate, which means they give answers that sound right but are factually wrong (Ji et al., 2023). It gets worse for temporal facts (events after the training cutoff), version-specific details (changes between software or policy editions), and population-specific claims (true for one group but not for another). One way to reduce this is multi-agent debate (MAD), where multiple independent agents give their own answers, argue with each other, and a judge picks the best-supported one (Du et al., 2023; Liang et al., 2023). Independent agents rarely make the same mistake in the same way, so if they disagree then at least one is probably wrong and the debate can find which one. The problem is that existing MAD systems debate every query, even when the agents already agree. If three agents get the same fact, running a full Propose-Critique-Revise-Judge cycle only adds latency (typically 4–8 seconds) and 10–15 LLM calls, and the answer is not any better. Evidence-MAD fixes this with an evidence gate, which is a pairwise Natural Language Inference (NLI) layer that compares the agent claims before anything else. If any two outputs contradict each other then the system goes to conflict classification and the full debate, and if they agree it directly returns an evidence-based answer. If the evidence is not enough or not clear, the system says that instead of faking confidence.

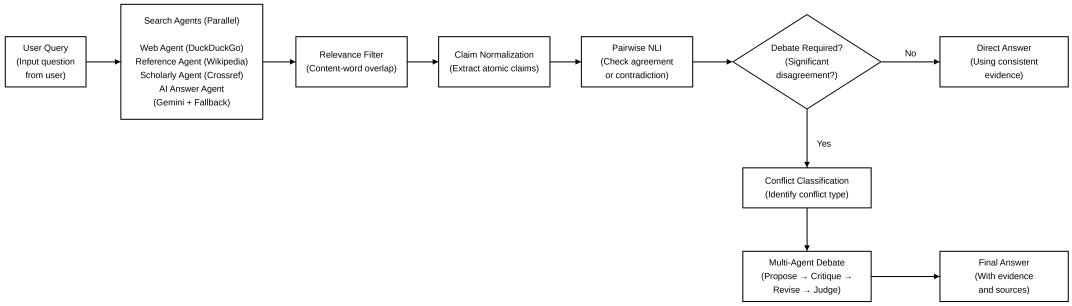


Figure 1.1: Overview of the Evidence-MAD Pipeline

Before the debate starts, the system checks what kind of conflict it is and puts it into one of five categories, FACTUAL, TEMPORAL, VERSION, CONTEXTUAL, or SOURCE. Each category has its own debate strategy, so the agents know exactly on which point to debate instead of debating blindly on every possible aspect. We use

four independent agents, a Web Agent that searches DuckDuckGo, a Reference Agent that queries Wikipedia, a Scholarly Agent that queries the Crossref academic database, and an AI Answer Agent powered by Google Gemini with Search grounding. The three search agents go through a two-stage normalization before the NLI comparison, so the claims are clean single sentences focused on the query instead of raw noisy snippets.

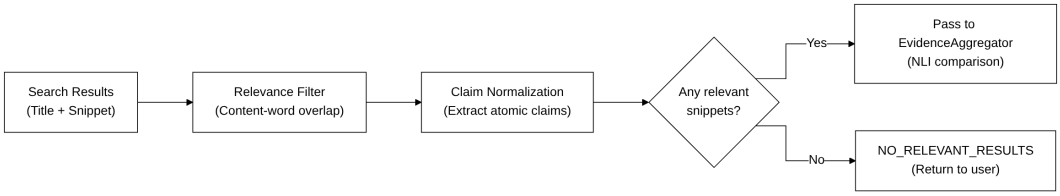


Figure 1.2: Evidence Gate Decision Flowchart

1.2 PROBLEM STATEMENT

The problem is simple, we have a natural language question and N independent retrieval agents, and we need to answer four things. First, how to find out in an efficient way whether the agent answers are really conflicting evidence or just different wording or retrieval noise. Second, how to avoid the cost of debate when the agents already agree, and still make sure the debate starts when there is a real disagreement. Third, when a debate is needed, how to change the resolution strategy according to the type of the conflict so it has a better chance of resolving it correctly. Fourth, how to handle the edge cases generic NLI models miss, such as entity-substitution conflicts (where one agent silently renames the requested entity) and explicit epistemic challenges (where one agent says a fact cannot be verified while another gives an affirmative answer).

Existing multi-agent debate work (Section 2.2) does not answer questions 2 and 3. Those systems either debate every time, which wastes cost, or use simple voting, which loses the structured argument. Evidence-MAD puts an NLI-based gate between retrieval and debate and adds conflict-type-aware strategies inside the debate.

1.3 OBJECTIVES

The objectives are: 1. design and implement an evidence-triggered gating mechanism using pairwise NLI that decides if a multi-agent debate is needed for a query. 2. Build a five-type conflict taxonomy (FACTUAL, TEMPORAL, VERSION, CONTEXTUAL, SOURCE) with an LLM-based classifier that puts each disagreement into one of them. 3. Implement conflict-type-specific debate strategies that guide the Proposer, Critic, and Reviser. 4. Extend the NLI layer with three deterministic query-aware guards for entity substitution, explicit epistemic challenge,

and target resolution mismatch, because generic NLI models fail on these. 5. Build a complete deployable system with a FastAPI REST backend, a browser-based frontend, and both offline (fixture) and live modes. 6. Evaluate everything with a controlled ablation study comparing six configurations, so the contribution of each component can be measured.

1.4 SCOPE OF THE PROJECT

The project covers open-domain factual question answering in English only. It retrieves from public web sources (DuckDuckGo, Wikipedia, Crossref) and uses the Google Gemini API for the AI answer agent. The evaluation dataset has only 15 samples across five conflict types, which is small and cannot represent all types of questions, so we do not claim more than that. It is not a commercial product and should not be used for safety-critical things like medical diagnosis or legal advice. We did not do full-page web content extraction (only titles and one-to-two sentence snippets are used), any model training or reinforcement learning, or cross-lingual question answering. The NLI part has two modes, LLM-backed NLI and a local HuggingFace transformer, but the transformer needs about 1.4 GB of disk space, so it cannot go to serverless deployment without extra infrastructure.

1.5 ORGANIZATION OF THE REPORT

Chapter 2 is the related work on multi-agent debate, NLI models, evidence retrieval, and conflict detection, ending with the research gap. Chapter 3 is the system design and architecture. Chapter 4 covers the four agents and their fallbacks. Chapter 5 covers the evidence pipeline from raw retrieval to relevance filtering, LLM normalization, and the aggregation decision. Chapter 6 covers the NLI detectors and the three query-aware guards, Chapter 7 the conflict classification, and Chapter 8 the debate lifecycle with Proposer, Critic, Reviser, and Judge. Chapter 9 is the API, frontend, and deployment. Chapter 10 is the evaluation and results, and Chapter 11 is the conclusion and future work.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

The related work falls in four areas, multi-agent systems and debate for LLMs, NLI models and how they are used for fact verification, evidence retrieval for open-domain QA, and conflict detection and classification. At the end we point out the gap that Evidence-MAD fills.

2.2 MULTI-AGENT SYSTEMS AND DEBATE

Using many agents to improve LLM output became popular after 2022. Society of Mind (Zhuge et al., 2023) split complex reasoning tasks between specialized agents and combined their outputs, AutoGen (Wu et al., 2023) gave a framework for conversations between LLM-based agents and improved coding and reasoning benchmarks, and ChatEval (Chan et al., 2023) used multi-agent argument for evaluation with agents as examiner and respondent. The closest one to our project is Du et al. (2023), who studied multi-agent debate for factual accuracy. On GSM8K and BioASQ, three agents debating for two rounds beat a single agent and ensembles. But they debate every question, which we think is not needed when the agents already agree. Liang et al. (2023) proposed a similar framework for reasoning tasks with comparable results.

Wang et al. (2023) studied self-consistency in chain-of-thought reasoning, where many reasoning paths are sampled and combined. It is not a debate system but it has the same idea that independent attempts show uncertainty. The difference is that it uses majority voting instead of structured argument and does not separate the types of disagreement. Madaan et al. (2023) proposed Self-Refine, where one model improves its own output through critique and revision. It looks like our Propose-Critique-Revise loop, but one model plays all roles and there is no external evidence or NLI-based conflict detection. Shinn et al. (2023) extended this with Reflexion, which adds episodic memory for multi-step tasks. The common problem in all of these is that none has an evidence gate. They run their debate or improvement loop for every query even if the first responses agree, so for simple queries where all agents agree anyway they waste computation. Evidence-MAD starts the debate only when a disagreement is detected.

Table 2.1: Summary of Related Work in Multi-Agent Debate Frameworks

System	Year	Evidence Gate	Conflict Types	Multi-Agent
Du et al. (MAD)	2023	No	None	Yes
Liang et al.	2023	No	None	Yes
Self-Refine	2023	No	None	Single
AutoGen	2023	No	None	Yes
ChatEval	2023	No	None	Yes
Evidence-MAD (Ours)	2025	Yes (NLI)	5 types	Yes

2.3 NATURAL LANGUAGE INFERENCE

NLI (also called recognizing textual entailment, RTE) finds the relation between a premise and a hypothesis, which can be ENTAILMENT (premise implies hypothesis), CONTRADICTION (premise contradicts hypothesis), or NEUTRAL (neither) (Dagan et al., 2006; Bowman et al., 2015). The standard benchmarks are the Stanford Natural Language Inference (SNLI) dataset (Bowman et al., 2015) and the Multi-NLI dataset (Williams et al., 2018). Transformers changed NLI a lot, BERT (Devlin et al., 2019) reached state-of-the-art results on MNLI by fine-tuning and models like RoBERTa (Liu et al., 2019) improved it more. The model we use, ynie/roberta-large-snli_mnli_fever_anli_R1_R2_R3-nli (Nie et al., 2020), is fine-tuned on SNLI, MultiNLI, FEVER-NLI, and three rounds of Adversarial NLI (ANLI) together, which makes it one of the most robust public NLI models. The ANLI rounds test adversarial robustness, which helps with the tricky entity-substitution cases in Chapter 6.

FEVER (Thorne et al., 2018) brought NLI into fact verification, where claims from Wikipedia are classified as SUPPORTED, REFUTED, or NOT ENOUGH INFO against retrieved evidence. This is close to our aggregation problem, since agent claims also need to be classified as supporting, contradicting, or inconclusive, but FEVER uses one curated source while we have multiple different retrieval agents. Some recent works also use LLMs directly for NLI instead of fine-tuned models. Sanh et al. (2022) showed that instruction-tuned models like T5 can do zero-shot NLI with reasonable accuracy, and Wei et al. (2022) showed that chain-of-thought prompting helps LLMs on reasoning-heavy NLI tasks. We support both a local transformer and an LLM-backed NLI mode, so inference cost can be balanced against deployment limits.

Table 2.2: Comparison of NLI Approaches Relevant to This Work

Model	Training Data	Approach	Key Strength
BERT-base	MNLI	Discriminative	Fast inference
RoBERTa-large	SNLI+MNLI+FEVER +ANLI	Discriminative	High accuracy
T5 (zero-shot)	C4 + instruction	Generative	No fine-tune needed
GPT-4 (prompted)	OpenAI pre-training	Generative	Handles nuance
LLMNLIDetector (Ours)	Groq endpoint	Prompted LLM	Query-aware guards

2.4 EVIDENCE RETRIEVAL APPROACHES

Open-domain QA systems mostly use dense retrieval (embedding-based similarity search over a pre-indexed corpus) or sparse retrieval (keyword-based search over live web or API sources). Dense approaches like DPR (Karpukhin et al., 2020) and RAG (Lewis et al., 2020) are popular in benchmarks because they match meaning precisely over large corpora, but they need a pre-indexed vector database and cannot get anything published after the index was built. We use sparse, live retrieval through public APIs (DuckDuckGo, Wikipedia, Crossref) because we want real-time, up-to-date information. The downside is that the snippets are noisier and shorter than passages from a curated index, which is why the two-stage normalization pipeline (Section 5.3) is needed.

WebGPT (Nakano et al., 2021) used LLM-assisted web browsing for open-domain QA and showed that real web retrieval gives better-grounded answers than the model's own knowledge. Perplexity.ai and the Google Search-grounded Gemini model in our AI Answer Agent also mix language generation with live search. Gou et al. (2023) proposed CRITIC, where an LLM checks its own claims with web search and keeps refining until the evidence supports it, which is similar to our normalization step where the LLM writes a claim that agrees with the retrieved snippets instead of its own knowledge. For scholarly retrieval, CrossRef (Van Eck and Waltman, 2010) and PubMed are the standard sources. Our ScholarlyAgent uses the Crossref REST API with DuckDuckGo as fallback and runs only for queries with academic vocabulary, so we skip academic search for general factual queries.

2.5 CONFLICT DETECTION AND CLASSIFICATION

Conflict detection between sources has been studied in knowledge base population (Nakashole and Mitchell, 2014), claim verification (Popat et al., 2018), and document summarization (Falke et al., 2019). Nakashole and Mitchell analysed conflicting statements in Freebase and found temporal conflicts (different time periods), definitional conflicts (different definitions of the same concept), and

measurement conflicts (different numerical values), and this directly influenced our five-type classification. Popat et al. (2018) did credibility-weighted conflict detection, where the reliability of a source decides how seriously its claims are taken. Our SOURCE conflict type partly follows this, if the disagreement is mainly about source authority (a blog post versus a peer-reviewed paper), the debate strategy tells the agents to check the provenance and credibility of the source instead of only the content of the claims.

The closest to our conflict classification is Schuster et al. (2019), who trained a classifier to separate the language phenomena that create apparent contradiction in NLI datasets (negation, antonymy, temporal reference, and so on). They wanted to understand NLI errors and not improve question answering, but their taxonomy overlaps a lot with our FACTUAL, TEMPORAL, and CONTEXTUAL categories. Pan et al. (2023) proposed FactScore, which breaks long LLM text into atomic claims and verifies each one against Wikipedia. It works at the claim level and not the agent level, but building verification around separate atomic claims is the same idea as the single-sentence claims of our EvidenceNormalizer.

2.6 RESEARCH GAP

Existing multi-agent debate systems (Du et al., 2023; Liang et al., 2023; Chan et al., 2023) run the full debate for every query whether the agents disagree or not, which wastes computation and ignores the signal that agent agreement gives. NLI-based fact verification systems (Thorne et al., 2018; Pan et al., 2023) compare claims against evidence but never use the NLI signal to gate a debate. Also, no multi-agent debate framework classifies the conflict type before debating. This matters because the strategy for a TEMPORAL conflict (check the dates) is very different from CONTEXTUAL (check whether the scopes are different populations) or SOURCE (check source authority), so one generic strategy for all types will probably give less accurate resolutions.

No earlier work looks at the query-aware NLI guard problem, where generic NLI models give NEUTRAL for claim pairs that are actually contradictory because of entity substitution or explicit epistemic challenge. We add three deterministic guards that fire before model inference for these cases, so there is a safety net that does not depend on how good the NLI model is. In summary, Evidence-MAD fills three gaps: (1) evidence-triggered gating to avoid unnecessary debate; (2) conflict-type classification with type-specific strategies; and (3) query-aware NLI guards for entity-level conflicts. These three together are the main technical contribution of this work.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

3.1 HIGH-LEVEL ARCHITECTURE

On disagreement, the ConflictClassifier gives a conflict type to the top-scoring contradicting pair, then the DebateManager runs up to MAX_DEBATE_ROUNDS cycles of Proposer, Critic, and Reviser with the conflict-type strategy text added in the Proposer prompt. After the debate finishes or stops early, the Judge reads the transcript and picks a winner, the AnswerGenerator makes the structured final answer, and the whole execution is saved as a JSON trace file.

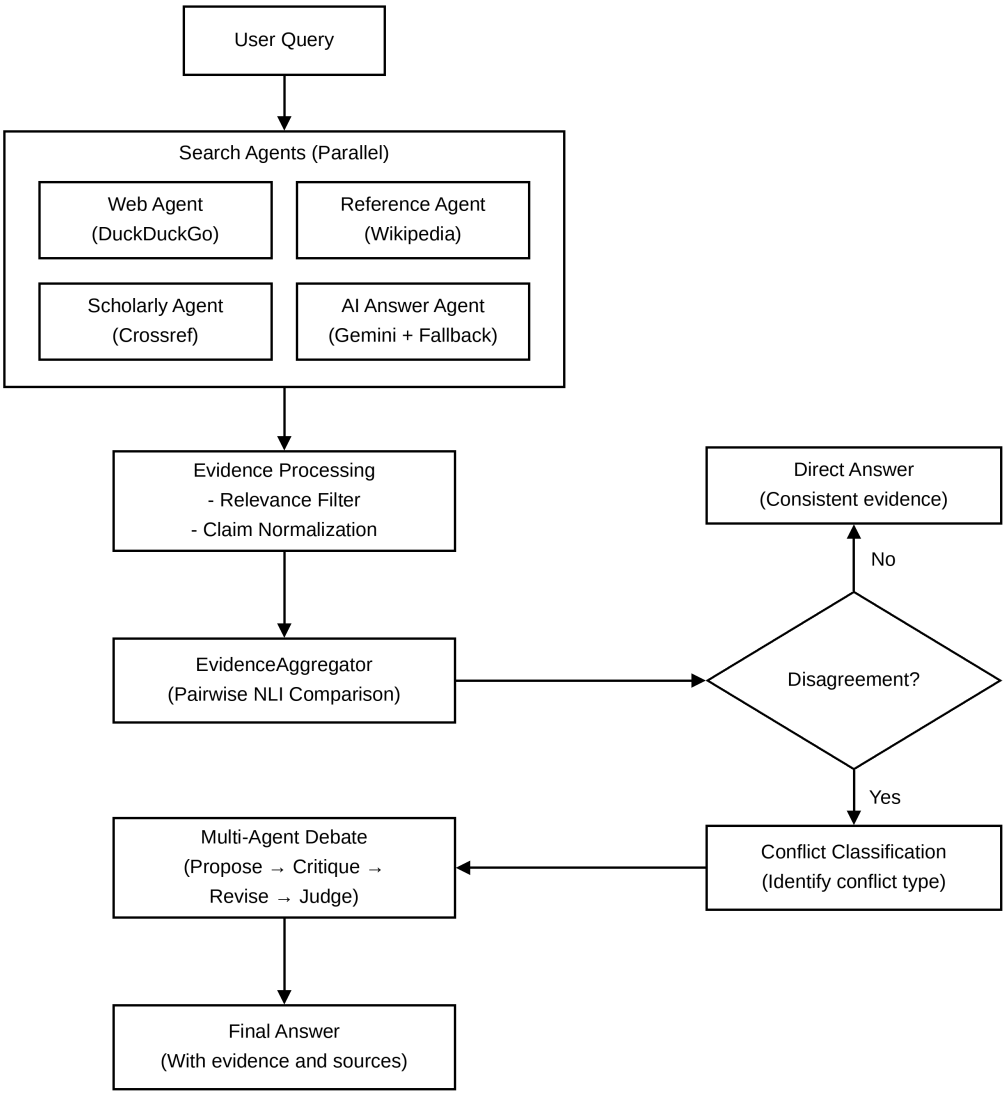


Figure 3.1: Evidence-Driven Multi-Agent Architecture

The system is a linear pipeline, implemented in MADSystem.solve() in app.py and fully synchronous, where every stage has a defined input and output type and can fail without bringing the whole pipeline down. The user submits a question, and four agents, Web, Reference, Scholarly, and AI Answer, retrieve evidence in parallel using a ThreadPoolExecutor. The three search agents go through a two-stage normalization to give one clean claim each, and the AI Answer Agent skips this since its output is already normalized. Then the EvidenceAggregator runs pairwise NLI on all the claims and gives one of four decisions, AGREEMENT, DISAGREEMENT, UNCERTAIN, or INSUFFICIENT. Three of these paths go directly to answer generation without any debate, and only DISAGREEMENT goes to the ConflictClassifier and DebateManager.

3.2 TWO-MODE OPERATION

There are two operating modes, strictly separated and controlled by the RunMode enum in run_mode.py. In FIXTURE mode (demo_mode=True) all agent evidence comes from data/fixtures.json with no live API call, the NLI detector uses exact-match lookup from the fixture table and all debate stages return fixed canned text, so it needs no API keys and is used for offline demos and unit tests. In LIVE mode (demo_mode=False) the full real pipeline runs and LLM_API_KEY must be set in the .env file.

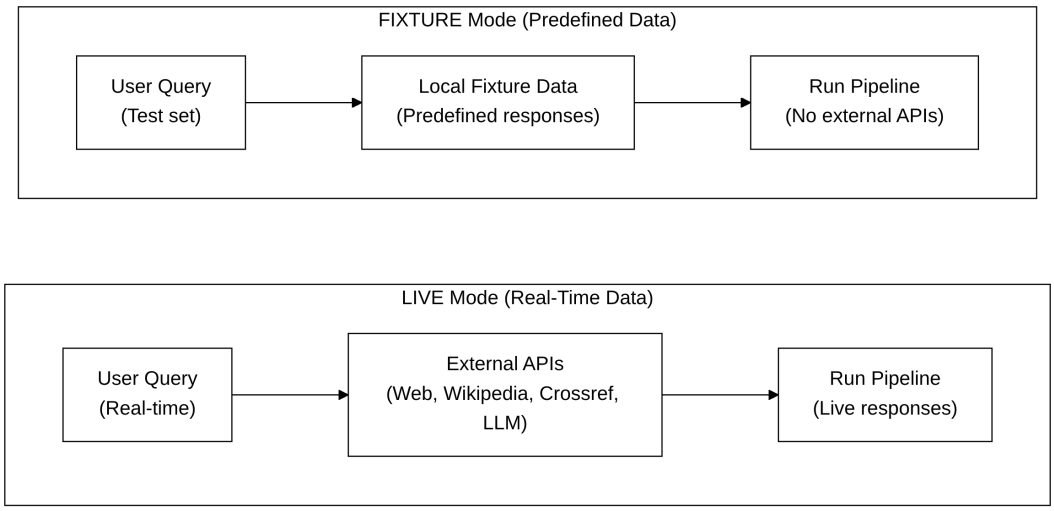


Figure 3.2: Fixture vs. Live Pipeline Modes

It uses LLMNLIDetector by default or NLIDetector (HuggingFace transformer) if NLI_MODE=transformers, the AI Answer Agent calls Gemini with Google Search grounding and falls back to the OpenCode CLI if Gemini fails, and the

search agents hit real HTTP endpoints with retry logic and exponential backoff. RunMode is passed into every component constructor and no component reads settings.demo_mode directly. We did this to prevent mode leakage, where a LIVE-mode component might fall back to fixture behavior by mistake, and it also makes unit testing easy because any component can be tested in FIXTURE mode without credentials.

3.3 COMPONENT DEPENDENCY MAP

The system has seven layers, Configuration, LLM Client, Agents, Evidence, Conflict, Debate, and Output, and each layer depends only on the layers below it, so there are no circular references. The Configuration layer (config.py) loads all parameters from the .env file using pydantic-settings, and the singleton through @lru_cache makes Settings created only once per process. The LLM Client layer (llm_client.py) wraps the Groq/OpenAI-compatible API and tracks call counts per role. The Agents layer has four independent retrieval agents and a QueryRouter for parallel dispatch.

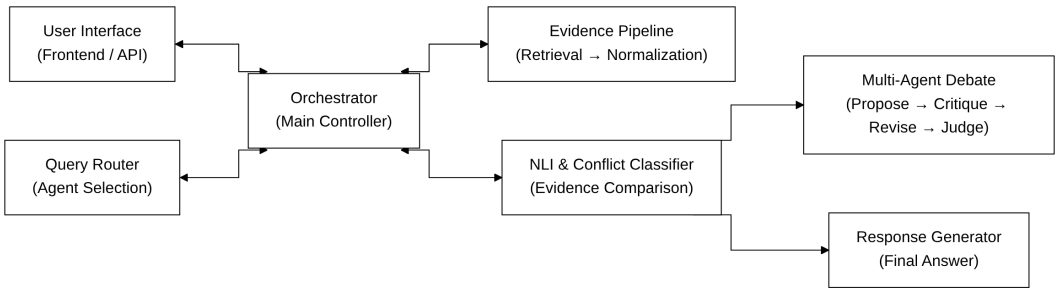


Figure 3.3: Component Dependency Map

3.4 CONFIGURATION AND ENVIRONMENT

All runtime configuration comes from environment variables in the .env file, validated by the Settings class in config.py, and the important ones are in Table 3.1.

validate() of Settings runs at startup in LIVE mode and raises ValueError if LLM_API_KEY is not set, so the system does not start without credentials and fail later on the first LLM call. get_settings() uses @lru_cache so the settings object is created only once per process.

Table 3.1: Key Configuration Parameters and Their Defaults

Environment Variable	**Default**	**Description**
LLM_API_KEY	(empty)	Groq / OpenAI-compatible key
LLM_BASE_URL	https://api.groq.com/openai/v1	LLM endpoint URL
LLM_MODEL	qwen/qwen3.8-27b	Model identifier
LLM_TEMPERATURE	0.0	Deterministic outputs
NLI_MODE	mock	mock = LLM NLI, transformers = HuggingFace
DISAGREEMENT_THRESHOLD	0.70	Min contradiction score for DISAGREEMENT
ENTAILMENT_THRESHOLD	0.70	Min entailment score for AGREEMENT
MAX_DEBATE_ROUNDS	3	Maximum debate iterations
DEMO_MODE	false	Use FIXTURE mode if true
GEMINI_API_KEY	(empty)	Google Gemini API key
GEMINI_MODEL	gemini-3.8-flash	Gemini model identifier

CHAPTER 4

AGENTS LAYER

4.1 AGENT REGISTRY AND BASE CLASS

All search agents inherit from BaseAgent, an abstract base class in agents/base_agent.py, which needs only one method, retrieve(query: str) → Evidence, returning an Evidence object with a list of raw search results. Each agent has a unique string agent_id, and the AgentRegistry is a global singleton dictionary mapping agent_id to the BaseAgent instance so the QueryRouter can find agents by identifier. AgentConfig is a Pydantic model that stores the agent_id, timeout in seconds, max_retries count, and an enabled flag, so with enabled=False an agent can be turned off through configuration without removing it from the codebase, which is useful for disabling the ScholarlyAgent when Crossref is down.

Table 4.1: Search Agent Summary

Agent Class	**Agent ID**	**Primary Source**	**Fallback**
WebAgent	web_agent	DuckDuckGo HTML	OpenCode CLI
ReferenceAgent	reference_agent	Wikipedia API	OpenCode CLI
ScholarlyAgent	scholarly_agent	Crossref REST	DuckDuckGo
AIAnswerAgent	ai_answer_agent	Google Gemini	OpenCode CLI

4.2 QUERYROUTER

The QueryRouter in agents/router.py sends the query to multiple agents in parallel using Python's concurrent.futures.ThreadPoolExecutor. The Web Agent and Reference Agent always run, and the Scholarly Agent runs only if a keyword heuristic matches. The heuristic scans the query for academic words such as study, trial, meta-analysis, doi, peer-reviewed, systematic review, rct, cohort, clinical, journal, research, findings, published, abstract, citation, and paper, and if any is found the ScholarlyAgent is dispatched with the others. In this way Crossref is called only when scholarly results are likely to be useful and unnecessary API calls are skipped for general factual questions.

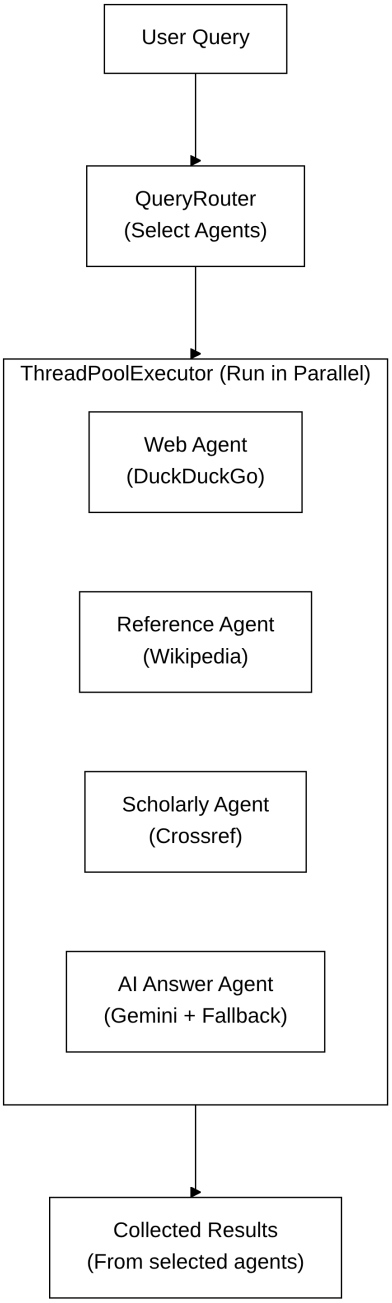


Figure 4.1: QueryRouter Parallel Agent Workflow

The AI Answer Agent is dispatched separately with its own single-thread executor that runs at the same time as the QueryRouter, so all four agents run in parallel and retrieval latency is lowest. The router collects results from all futures with a timeout, and any agent that times out or raises an exception is marked RETRIEVAL_FAILED and the rest continue.

4.3 WEB AGENT

The WebAgent (agents/search_agent.py) uses the HTML interface of DuckDuckGo at https://html.duckduckgo.com/html/, because unlike the JavaScript-heavy main interface it can be scraped with simple HTTP requests and regex matching. It takes the titles from elements with class result__a and the snippets from class result__snippet, and since DuckDuckGo encodes redirect URLs as /l/?uddg=<encoded>, it decodes them with urllib.parse.parse_qs to get the actual destination URL. Retry logic uses exponential backoff, up to three attempts with delays of 2, 4, and 8 seconds on HTTP 429 (rate limit), 5xx (server error), and connection timeout. If all retries fail, _opencode_web_fallback() sends the query to an OpenCode CLI model and returns synthetic search results, which carry the same web_agent identity, so the NLI and debate layers never see provider-level information and the abstraction boundary stays clean.

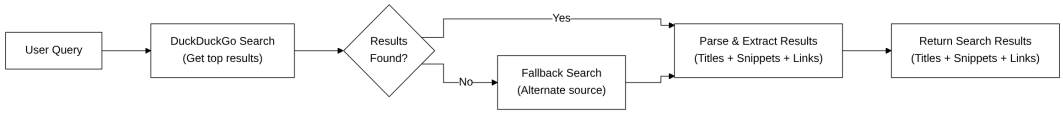


Figure 4.2: Web Agent Search Workflow

4.4 REFERENCE AGENT

The ReferenceAgent queries the Wikipedia MediaWiki API at https://en.wikipedia.org/w/api.php with action=query, list=search, and srsearch=<query>, and gets up to five results. For each result it fetches the page extract (summary paragraph) with a second call using action=query and prop=extracts, and cuts it to the first 500 characters so the claim stays within the snippet length the normalizer expects. Wikipedia rate limits can give HTTP 429 when queries come quickly one after another, so the agent waits with exponential backoff and retries up to three times, and if the limit continues it falls back to the OpenCode CLI using OPENCODE_WIKI_MODEL, a lighter model made for reference-style retrieval. In this way the Reference Agent always gives a claim to the evidence pool and we keep $N \geq 2$ usable agents in most cases.

4.5 SCHOLARLY AGENT

The ScholarlyAgent queries the Crossref REST API at https://api.crossref.org/works?query=<query>&rows=5, which returns JSON with a message.items list where each item has title, DOI, abstract, author, and published date. The agent takes the title and abstract and cuts long abstracts to 300 characters. If Crossref gives no results or an HTTP error, it falls back to DuckDuckGo search with the word research added to the query so the results lean to academic sources. Crossref results are metadata of papers and not direct answers to factual questions, so the LLM normalization step (Section 5.3) matters a lot here because it has to pull a claim-relevant statement out of abstracts written for specialists.

4.6 AI ANSWER AGENT

The AIAnswerAgent (agents/ai_answer_agent.py) is different from the three search agents because it does not retrieve results and then normalize them. It queries the Google Gemini API with Google Search grounding enabled and directly produces one normalized claim, since Gemini with Search grounding can access real-time web results while generating, so retrieval and normalization happen in one step. The prompt (PROMPT_TEMPLATE) clearly says: "Preserve every named entity exactly as written in the question: never silently correct, replace, or assume that a similarly named paper, product, person, or work is the same entity." This is there for the entity-substitution problem in Section 6.3. The claim is trimmed to at most 600 characters at a sentence boundary by the _short() helper because Gemini often gives multi-paragraph responses, and the full response is kept in metadata['full_answer'] for the final answer generation.

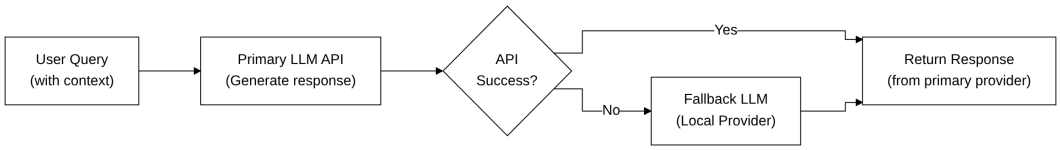


Figure 4.3: AI Answer Agent Flowchart

CHAPTER 5

EVIDENCE PIPELINE

5.1 EVIDENCE FLOW OVERVIEW

The evidence pipeline turns raw search results from the three search agents into clean, normalized, query-focused claims for NLI. The AI Answer Agent skips the first two stages because its output is already one synthesized claim, so for the search agents there are three stages, relevance filtering, LLM normalization, and NLI aggregation. After retrieval each agent has raw results with a title and snippet, the relevance filter removes results whose content words do not overlap enough with the query, the remaining ones go to the LLM normalizer which produces one NormalizedClaim with structured fields (text, date, version, scope, source), and then the EvidenceAggregator runs pairwise NLI on all the agent claims to give the final EvidenceDecision.

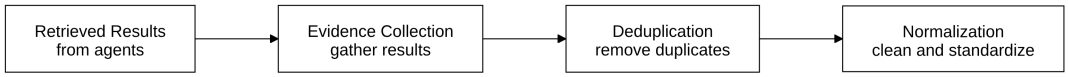


Figure 5.1: Evidence Flow: Retrieval to Normalization

5.2 RELEVANCE FILTER

The relevance filter is the _is_relevant() function in evidence/normalizer.py. It finds the content-word overlap between the query and the title plus snippet of a result, where content words are all words not in a hardcoded stopword list of about 50 common English words. Numbers always count as content words whatever their length, because year strings like 2020 are often the most useful feature for temporal queries. The min_overlap threshold is 1, so at least one content word of the query must be in the title or snippet, and for very short queries with two or fewer content words it is always 1 so the filter does not remove too much. Results that fail are counted in the filtered_out_count field of AgentEvidence, which shows in the execution trace and is used by the evaluation framework for the average irrelevant evidence rate.

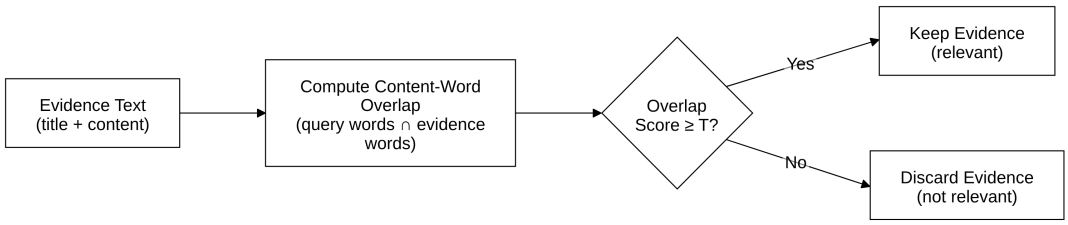


Figure 5.2: Content-Word Overlap Relevance Filter

If all results of an agent are filtered out, its status is set to EvidenceStatus.NO_RELEVANT_RESULTS. This is different from EvidenceStatus.RETRIEVAL_FAILED, because the first means the agent retrieved results but none were relevant and the second means it could not retrieve at all because of a network error, API failure, or timeout. Both remove the agent from the NLI comparison but they are reported separately for diagnosis.

5.3 LLM NORMALIZATION

The surviving results go to the LLM normalizer with a prompt that has the original query and a numbered list of titles and snippets, and asks for one short answer sentence. It also extracts date (year), version, and scope (population or platform) along with the answer, and these fields are very important later. The date field helps the NLI pattern detector find temporal conflicts (one claim says 2019 and another says 2020), version helps version conflict detection, and scope helps contextual conflict detection (one claim is for adults and another for children). Without them, the pattern-based NLI fallback would have to pull dates and versions from free text, which makes more errors. The normalizer also does meta-claim detection. Sometimes the LLM returns something like 'The snippets do not contain information about this topic' instead of an answer, so it checks for ten meta-language patterns such as do not contain, no information, no relevant, unable to find, and not mentioned. If any appears, the result is treated as NO_RELEVANT_RESULTS and the meta-claim is not stored, because if it is compared against real claims in NLI it will always give false contradictions.

5.4 EVIDENCE AGGREGATOR

The EvidenceAggregator (evidence/aggregator.py) runs NLI on every unordered pair of usable agents, so three usable agents give three comparisons (A-B, A-C, B-C). The decision rule is asymmetric, one contradicting pair is enough for disagreement (conservative sensitivity) but agreement needs all pairs to confirm entailment (conservative specificity). This is on purpose, because in evidence-

triggered debate a false negative (missing a real disagreement and returning a wrong agreed answer) is worse than a false positive (starting an unnecessary debate when the agents actually agree). So if even one pair shows contradiction above the DISAGREEMENT_THRESHOLD, the system goes to debate instead of silently hiding the conflict.

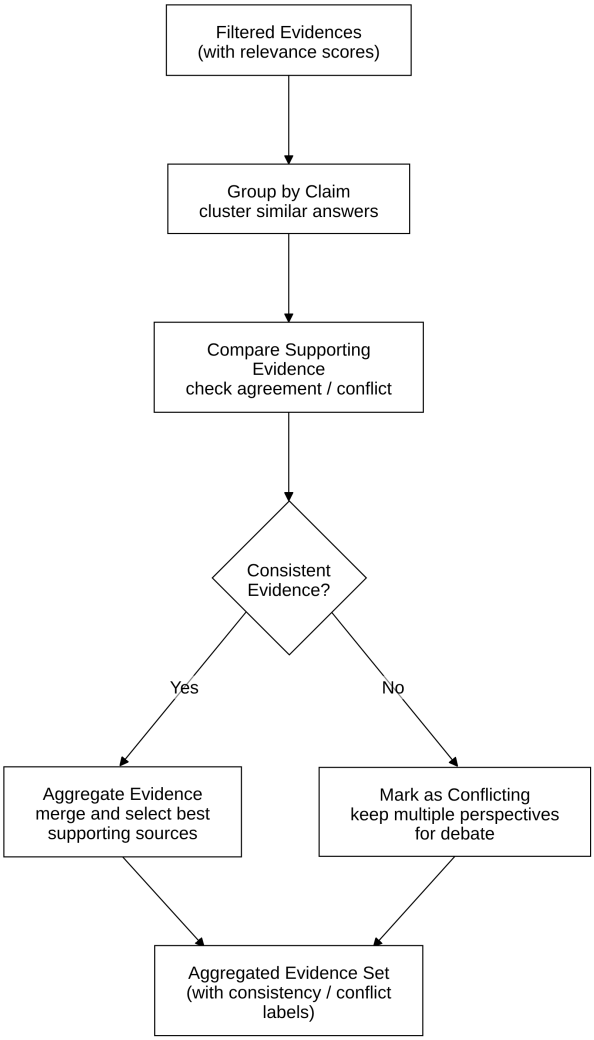


Figure 5.3: Evidence Aggregation Decision Flowchart

The aggregator records the top_conflict, which is the agent pair with the highest contradiction score, and its claims become claim_a and claim_b in the ConflictClassifier and DebateManager, so the debate is on the strongest conflicting evidence and not a random pair. If NLI raises an NLIError for a pair, that pair is recorded with all scores 0.0 and label NEUTRAL, so one failing NLI call cannot crash the aggregation. The formal decision rule is: if the count of usable agents is less than or equal to one, return INSUFFICIENT; else for each pair compute NLI scores;

if any pair has contradiction_score greater than or equal to DISAGREEMENT_THRESHOLD return DISAGREEMENT; else if all pairs have entailment_score greater than or equal to ENTAILMENT_THRESHOLD return AGREEMENT; else return UNCERTAIN.

Table 5.1: EvidenceStatus Values and Their Meanings

EvidenceStatus	**Meaning**	**Included in NLI?**
OK	Retrieval succeeded and claim normalized	Yes
NO_RELEVANT_RESULTS	Retrieval ok but no relevant snippets	No
RETRIEVAL_FAILED	Network/API error during retrieval	No

CHAPTER 6

NLI AND CONFLICT DETECTION

6.1 NLI DETECTOR CLASSES

There are three NLI detector classes in evidence/disagreement_detector.py, one for each runtime environment, and all have the same interface, predict_for_query(query, premise, hypothesis), which returns a PairwiseRelation object with label, contradiction_score, entailment_score, and neutral_score. MockNLIDetector is for FIXTURE mode, it does exact-match lookup in a fixture table keyed by (premise, hypothesis) pair identifiers and falls back to _pattern_predict() if there is no entry, and it never calls any LLM or model so FIXTURE mode stays fully offline. LLMNLIDetector is for LIVE mode with NLI_MODE=mock (the default), and the name is confusing, it is called mock because it mocks the transformer, but it actually uses the real LLM endpoint (Groq/OpenAI-compatible), sending a structured prompt with the query, claim A, and claim B and asking for JSON with label and score fields, and it falls back to _pattern_predict() if the LLM fails. NLIDetector is for LIVE mode with NLI_MODE=transformers, it loads ynie/roberta-large-snli_mnli_fever_anli_R1_R2_R3-nli through HuggingFace transformers and runs inference in both directions, claim A as premise and B as hypothesis and then reverse. The final scores use the maximum of both contradiction scores and the minimum of both entailment scores, which is a conservative combination that is sensitive to asymmetric contradictions.

Table 6.1: NLI Detector Class Comparison

Class	**Mode**	**Backend**	**Fallback**
MockNLIDetector	FIXTURE	Fixture table	_pattern_predict()
LLMNLIDetector	LIVE + NLI_MODE=mock	LLM (Groq)	_pattern_predict()
NLIDetector	LIVE + NLI_MODE=transformers	HuggingFace RoBERTa	None (raises)

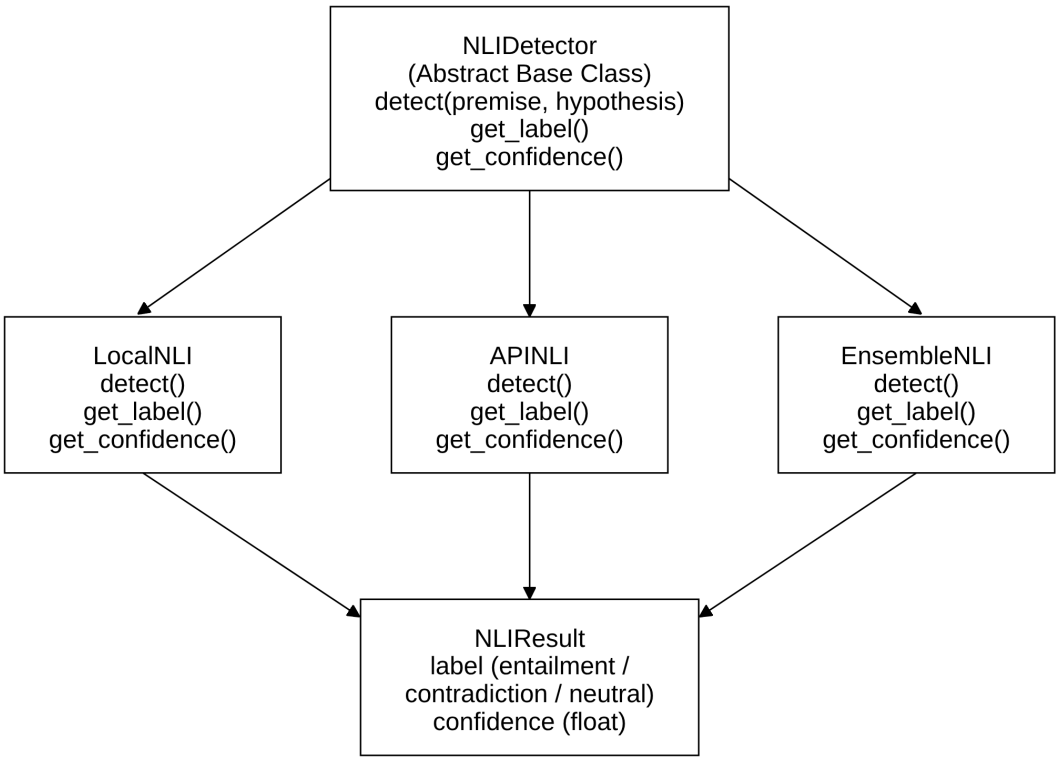


Figure 6.1: NLI Detector Class Hierarchy Diagram

6.2 QUERY-AWARE GUARD CHAIN

All three detector classes apply a chain of three deterministic query-aware guards before any NLI model is used. They are regular expression matchers for specific cases where generic NLI models are known to fail, and if any guard fires its result is returned immediately without calling the NLI model, so the behavior is the same whichever detector class is used. They run in order, entity substitution, then explicit challenge, then target resolution, and only if none fires the system goes to model-based inference. The order follows how specific they are, entity substitution is the most specific (it needs a paper-naming pattern), explicit challenge is broader (any epistemic rejection against an affirmative answer), and target resolution is the most complex (it needs entity comparison across claims).

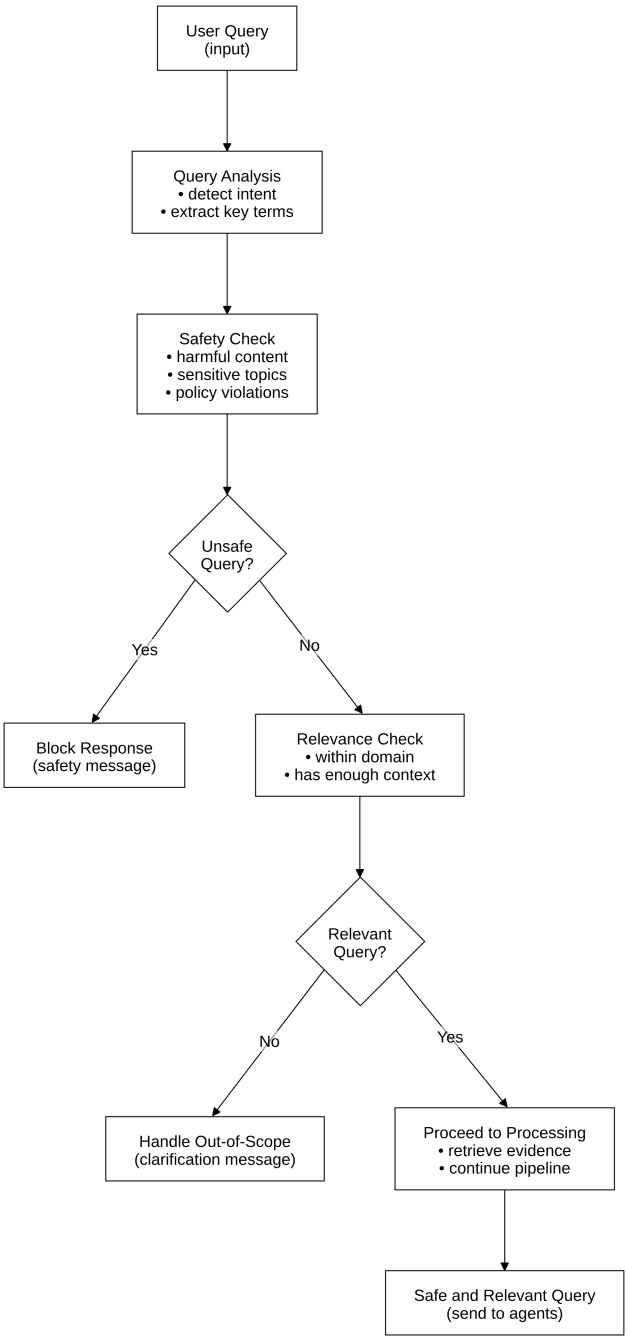


Figure 6.2: Query-Aware Guard Chain Flowchart

6.3 ENTITY SUBSTITUTION GUARD

The entity substitution guard looks for one claim naming a paper (or another published work) with a title in quotation marks while another claim clearly rejects that such a paper exists. This covers the case where a search agent returns results for a similarly titled paper (an autocorrection) while the AI Answer Agent correctly says the paper that was actually asked about does not exist. It uses two regular expressions, the rejection pattern matches phrases like no such paper exists, does not exist, no

record of the paper, and cannot find the article, and the work-naming pattern matches phrases like paper titled followed by a quoted string, publication titled, and article titled. If rejection fires in one claim and work-naming in the other, the guard returns CONTRADICTION with confidence 0.91. We chose 0.91 to be above the DISAGREEMENT_THRESHOLD of 0.70 so the debate triggers, but below 1.0 because the guard is heuristic and can sometimes fire wrongly. It is asymmetric, it fires only when one claim has a rejection and the other has a positive identification, and not when both have rejections, since that means both agree it does not exist.

6.4 EXPLICIT CHALLENGE GUARD

The explicit challenge guard is the most general one and does not depend on the domain. If one agent clearly says the requested fact cannot be verified, has no reliable support, or does not exist, and another agent gives an affirmative answer about the same subject, the two claims are in contradiction. The challenge pattern matches phrases like no record, no known record, no official record, does not exist, doesn't exist, cannot verify, can't verify, unable to verify, unverified, unsupported, unsubstantiated, nonexistent, not verified, not verifiable, not supported, and not documented, and a second check makes sure the challenging claim has at least one content word from the query (or a deictic pronoun like it or this), so a random disclaimer in unrelated text cannot trigger it. This matters because LLMs sometimes show uncertainty by saying a fact is unverifiable while a search engine returns a page that mentions the topic (even if unreliable), and a generic NLI model may label these as NEUTRAL because the wording is very different. With the guard these cases always trigger a debate, which is correct, because then the judge can weigh the source quality of the affirmative answer.

6.5 TARGET RESOLUTION GUARD

The target resolution guard handles the most complex failure, where one agent rejects the queried entity by name (e.g., no known paper titled Wipro is All U Need) while another agent positively identifies a different but similarly named entity (e.g., the paper Attention is All You Need was published in 2017). A generic NLI model may call these ENTAILING because they share a year and both are about machine learning papers, and miss that they are about different entities. The guard converts the query and the claims into lowercase alphanumeric tokens, extracts the rejected titles (after phrases like no known paper titled) and the positively asserted titles (quoted titles in claims without rejection language), and if a rejected title overlaps with the query and a positively asserted title is different from it, it returns CONTRADICTION

with confidence 0.93, a bit higher than the entity substitution guard because target resolution checks are more precise.

6.6 TRANSFORMER-BASED NLI

With NLI_MODE=transformers, the NLIDetector class loads the HuggingFace text-classification pipeline with the ynie/roberta-large model, a RoBERTa-large fine-tuned on SNLI (Bowman et al., 2015), MultiNLI (Williams et al., 2018), FEVER-NLI (Thorne et al., 2018), and three rounds of Adversarial NLI (Nie et al., 2020), which makes it much more robust to adversarial examples than models fine-tuned on one dataset. The pipeline uses top_k=None so it returns scores for all three labels (ENTAILMENT, NEUTRAL, CONTRADICTION) and not only the top one, because the aggregator needs all three scores to check the DISAGREEMENT and AGREEMENT thresholds. The input uses the HuggingFace NLI convention of separate text and text_pair fields, so the separator token goes between premise and hypothesis the same way as in fine-tuning.

CHAPTER 7

CONFLICT CLASSIFICATION

7.1 CONFLICT CLASSIFIER

After the EvidenceAggregator returns DISAGREEMENT, the ConflictClassifier (conflict/classifier.py) finds the type of the conflict. It takes the texts of the two most contradicting claims and the original query and returns a ClassificationResult with the conflict type, confidence score, explanation, and a ComponentStatus that says whether the LLM worked or the pattern fallback was used. ComponentStatus is useful for evaluation because it lets us measure LLM classification and pattern fallback accuracy separately. If the LLM API is down during an evaluation run, all classifications fall back to pattern matching and the results cannot be compared with a run where the LLM was available, so ComponentStatus shows this difference instead of hiding it.

7.2 FIVE-TYPE TAXONOMY

We made the five types to cover the most common kinds of factual disagreement in open-domain QA, by studying the literature (Nakashole and Mitchell, 2014; Schuster et al., 2019) and iterating on the conflicts we saw while testing. Table 7.1 gives each type with its definition and an example.

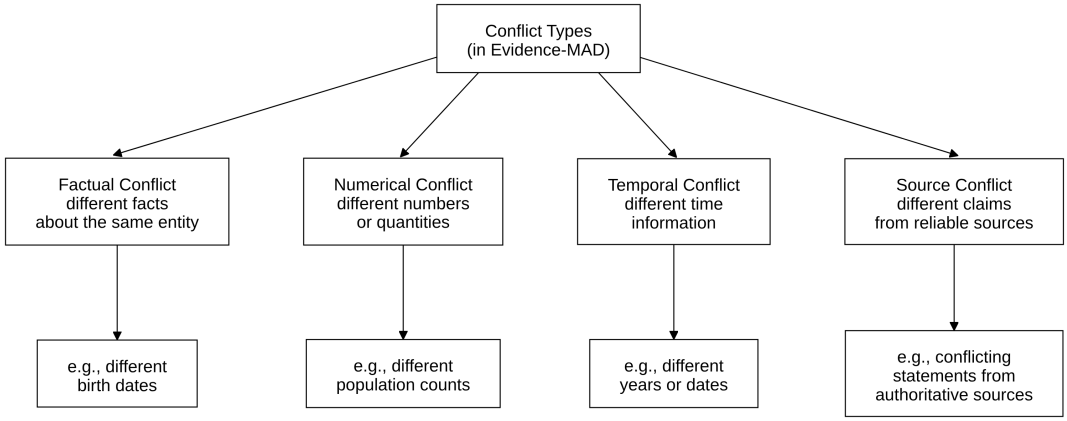


Figure 7.1: Conflict Types: Evidence-MAD Taxonomy

Table 7.1: Conflict Type Definitions and Representative Examples

Conflict Type	**Definition**	**Example**
FACTUAL	Mutually incompatible factual claims at same time/context	Agent A: water boils at 100°C. Agent B: water boils at 212°F at sea level
TEMPORAL	Claims differ because they refer to different time points	Agent A: policy enacted in 2019. Agent B: policy enacted in 2020
VERSION	Claims refer to different software/document versions	Agent A: Python 3.10 supports match. Agent B: Python 3.9 does not
CONTEXTUAL	Claims differ due to different conditions/populations	Agent A: drug effective for adults. Agent B: drug not effective for children
SOURCE	Disagreement driven by source authority/reliability difference	Agent A (blog): supplements cure disease. Agent B (journal): no evidence found

The LLM classifier gets a prompt with these definitions and examples for each type and classifies the conflict between the two claims. It returns JSON with a type field (one of the five types or UNKNOWN) and a confidence field, and if it returns UNKNOWN or confidence below 0.5, the system goes to the pattern-based fallback instead of accepting it.

7.3 PATTERN-BASED FALLBACK

The pattern-based fallback uses a rule hierarchy in order. First, if both claims have different version number patterns (e.g., v3.10, Python 3.13, v2), it is VERSION. Second, if both have different year strings (four-digit numbers), it is TEMPORAL. Third, if the claims mention population or platform qualifiers from hardcoded groups (adults vs. children, macOS vs. Windows, safe vs. dangerous), it is CONTEXTUAL. Fourth, if they have source quality indicators (blog vs. paper, unofficial vs. official, unverified vs. peer-reviewed), it is SOURCE. If none fires, the default is FACTUAL, which is reasonable because plain factual disagreements are the most common case. The fallback is conservative on purpose, it will misclassify some conflicts (mainly edge cases at the boundary between types) but its false positive rate is very low for the types it detects reliably (TEMPORAL and VERSION especially), and the ComponentStatus field lets the evaluation measure the accuracy gap between the LLM path and the fallback path.

CHAPTER 8

DEBATE LIFECYCLE

8.1 DEBATE MANAGER

The DebateManager (debate/manager.py) controls the Propose-Critique-Revise cycle. run_debate() takes the classified conflict, the two claims, the query, and the agent evidence, runs up to MAX_DEBATE_ROUNDS iterations (default 3), and checks early stop conditions after each stage, so the debate does not go on endlessly. Each round gives a DebateRound object with the proposer, critic, and reviser arguments, stored as DebateArgument objects with round number, role, content, evidence references, conflict type, strategy used, and a 200-character preview of the strategy text. The preview is saved in the trace file for debugging, so we can verify that the correct conflict-type strategy was added to the proposer prompt.

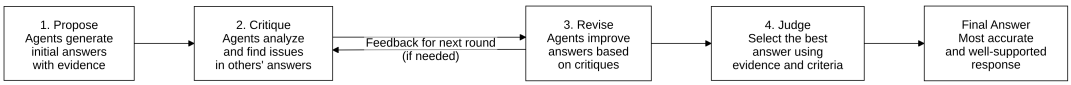


Figure 8.1: Debate Lifecycle Infographic Flowchart

LLM failures are handled safely. If any stage (Proposer, Critic, or Reviser) raises an LLMError the debate stops immediately, with status failed if it happens in round 1 and partial if in a later round, and failed_stages records which stages failed. The Judge is called only if the status is completed and never on a failed or partial debate, so the system does not give a wrong verdict from an incomplete argument.

8.2 PROPOSER

The Proposer (debate/proposer.py) generates the first position in each round, and its prompt is the most complex in the system because it has to include the conflict-type strategy along with the claim information. It receives the query, claim A text, claim B text, and the conflict type, and has to produce a structured argument with five sections: POSITION, EVIDENCE REVIEW, REASONING, SOURCE BASIS, and CONCLUSION. The strategy is different for each type. For TEMPORAL conflicts it tells the proposer to check the recorded date of each claim and leave out claims dated outside the query period. For VERSION conflicts it tells the proposer to check if both claims are about the same version, and if not, to report both as correct for their own version. For CONTEXTUAL conflicts it focuses on whether the claims are for different populations or platforms, which can remove the contradiction instead

of choosing one side. The Proposer also gets metadata from the normalized claims, the date field of each claim and a QUERY PERIOD field from the query if a year is present for TEMPORAL, the version field for VERSION, and the scope field for CONTEXTUAL, so it can make specific evidence-based arguments instead of generic position statements. Before the output goes to the critic, is_valid_argument() rejects arguments shorter than 50 characters or beginning with Demo response, so very short or canned LLM responses do not enter the debate.

8.3 CRITIC

The Critic (debate/critic.py) gets the proposer argument and judges it on the evidence. If the argument is well-reasoned and well supported, the critic signals an early stop by including SUBSTANTIVE_OBJECTION: false in its response, and if not, it writes a substantive objection pointing out specific problems in the reasoning or evidence references. The no_objection check is a case-insensitive regex match for SUBSTANTIVE_OBJECTION: false in the critic content, and it also checks the strategy_used field of the DebateArgument for the same string. If either has the signal, the debate ends with stop_reason=no_objection and status=completed, and the judge is called immediately on the unchallenged proposer argument. This path matters when the proposer correctly finds that the conflict goes away after checking, like a CONTEXTUAL conflict where the two claims are about different populations and both are correct in their scope, because then a reasonable critic agrees and the early stop saves an unnecessary round.

8.4 REVISER AND EARLY STOPPING

The Reviser (debate/reviser.py) combines the proposer position and the critic objections into a better argument. It receives the full debate context from all previous rounds and has to answer the critic concerns while keeping the best-supported position from the evidence. After the Reviser, the DebateManager checks convergence using Jaccard similarity between the current and previous revision, calculated over word sets as intersection of tokens divided by union, and if the score is 0.95 or higher the revision is treated as converged and the debate stops with stop_reason=converged. The 0.95 threshold is strict on purpose, because two arguments sharing 95% of their tokens are basically the same argument with small phrasing differences, so more rounds will not improve the resolution, and the strict threshold makes sure convergence fires only when the debate has really stopped improving and not when two rounds just share vocabulary.

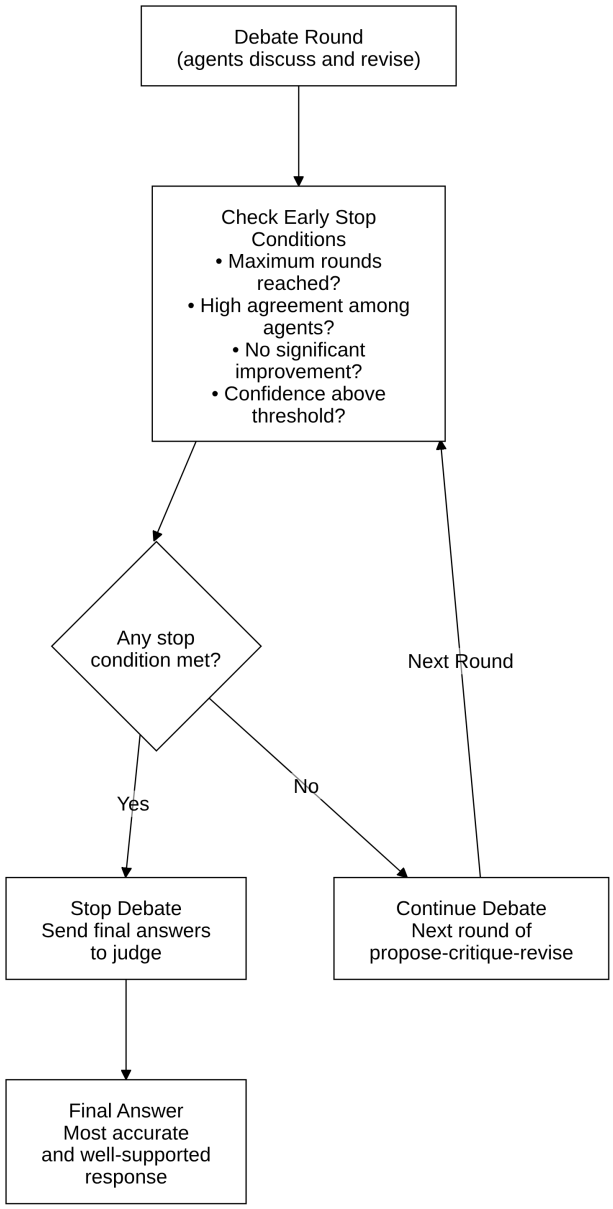


Figure 8.2: Early Stop Debate Flowchart

Table 8.1: Debate Stop Reason Summary

Stop Reason	**Condition**	**Debate Status**
converged	Jaccard(revision_n, revision_{n-1}) ≥ 0.95	completed
no_objection	Critic returns SUBSTANTIVE_OBJECTION: false	completed
max_rounds	MAX_DEBATE_ROUNDS iterations completed	completed
stage_failure	Proposer/Critic/Reviser raises LLMError	failed / partial

8.5 JUDGE

The Judge (debate/judge.py) is called only when debate_outcome.status == completed. It receives the full transcript (all rounds with proposer, critic, and reviser arguments), the structured claim information (text, date, version, scope, source URL) of both claims, and the conflict type, and produces a JudgeResult with a winner (candidate_1 or candidate_2), an evidence score, a source score, a reasoning score, and an overall confidence. Its prompt clearly says to give more importance to evidence quality than rhetorical skill, so a well-argued position backed by a low-quality source should lose against a less elegant argument backed by a primary, authoritative source, and the three separate scores let the evaluation check which one influenced the decision the most. The Judge also never makes up a verdict. If its LLM call fails, judge_result is set to None, the AnswerGenerator detects this and generates an unresolved conflict answer instead of claiming a winner, and a null judge_result is always shown clearly in the response and never silently replaced with a default value, so the system does not show a hallucinated verdict as if it was a real result.

CHAPTER 9

API, FRONTEND AND DEPLOYMENT

9.1 FASTAPI APPLICATION

The REST API is a FastAPI application (api.py) version 2.0.0. We chose FastAPI over Flask or Django for three reasons, its Pydantic-based validation is consistent with the rest of the codebase. There are four endpoints, GET / returns a name/version/status JSON, GET /health returns {"status": "healthy"}, POST /api/solve is the main endpoint that takes a JSON body with query and an optional demo_mode field, and POST /solve is an alias kept for backward compatibility. CORS allows requests from http://localhost:3000 and http://127.0.0.1:3000, which match the frontend development server port.

Table 9.1: Key Fields in the /api/solve Response Schema

Field	**Type**	**Description**
query	str	The user's question
answer	str	The final synthesized answer
evidence_state	str	agreement | disagreement | uncertain | insufficient
debate_triggered	bool	Whether the debate pipeline ran
conflict_type	str | null	factual | temporal | version | contextual | source
agent_evidence	list	Per-agent status, claim, source
pairwise	list	NLI scores for each agent pair
debate_rounds	list	Per-round proposer/critic/reviser content
judge	object | null	Winner, scores, reason
sources	list	Deduplicated {url, title} source list
metrics	object	Latency, LLM call counts, stop reason

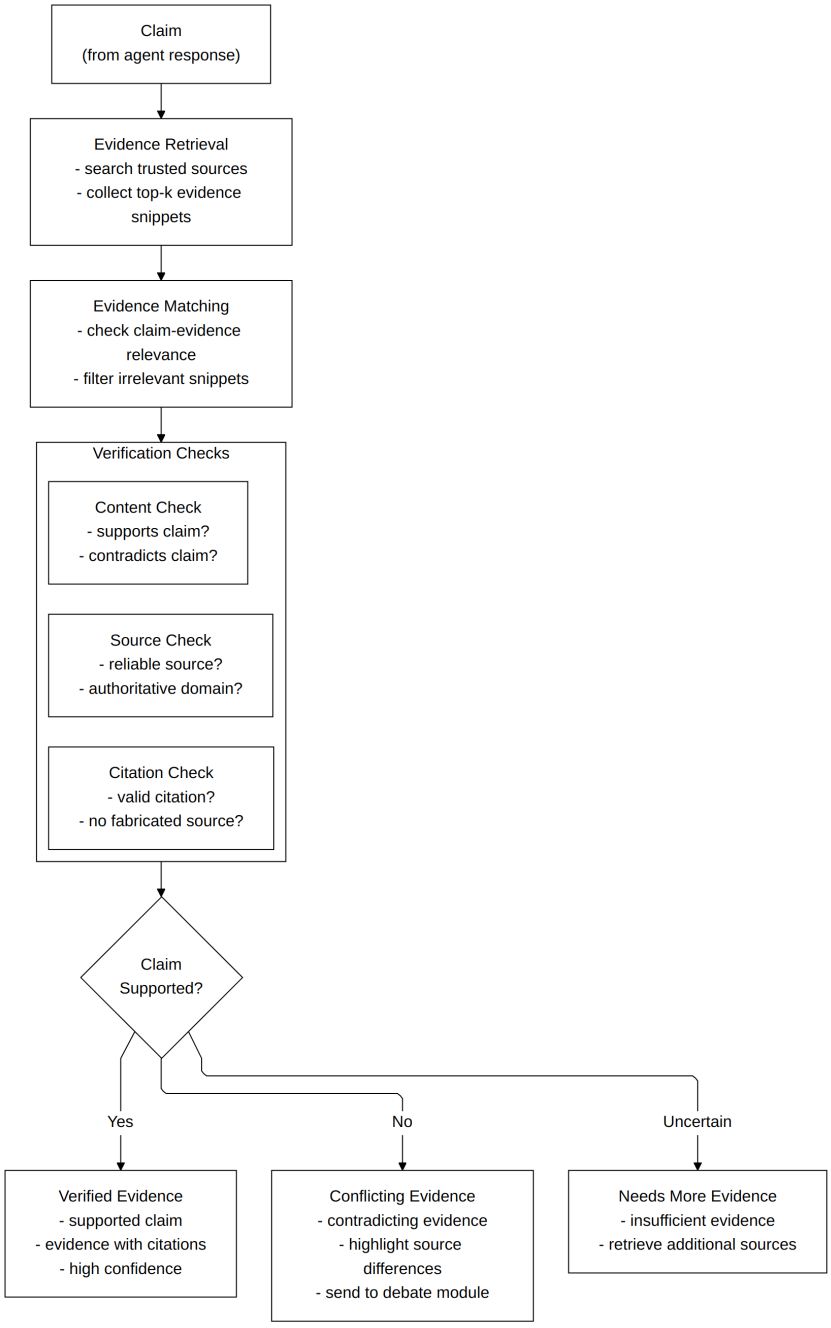


Figure 9.1: Evidence Verification Pipeline Diagram

MADSystem.solve() is synchronous (it uses ThreadPoolExecutor internally for parallelism), so the async FastAPI endpoint wraps it with loop.run_in_executor(None, lambda: sync_solve(...)). This stops the pipeline from blocking the event loop, so the server can handle other requests while a long solve is running.

9.2 FRONTEND

The frontend is a single HTML file (frontend/index.html) with no build step, no Node.js, and no npm, served by the built-in HTTP server of Python with python -m http.server 3000 --bind 127.0.0.1 --directory frontend, so it runs anywhere that has Python 3. The UI has a query text field and a Demo mode toggle, and after submission it shows the agent evidence cards with status, claim, and source of each agent, a pairwise NLI scores table, an evidence decision badge (color-coded: green for AGREEMENT, red for DISAGREEMENT, yellow for UNCERTAIN, grey for INSUFFICIENT), the conflict type when a disagreement is detected, the full debate transcript with proposer/critic/reviser sections for each round, the judge verdict with scores, the final answer, and a metrics panel with latency and LLM call counts. It talks to the backend only through /api/solve using the built-in fetch API with JSON bodies, with no WebSocket or streaming, so it waits for the complete response before showing anything. For live-mode queries the UI can look unresponsive for 4–8 seconds, so a loading spinner is shown during that time.

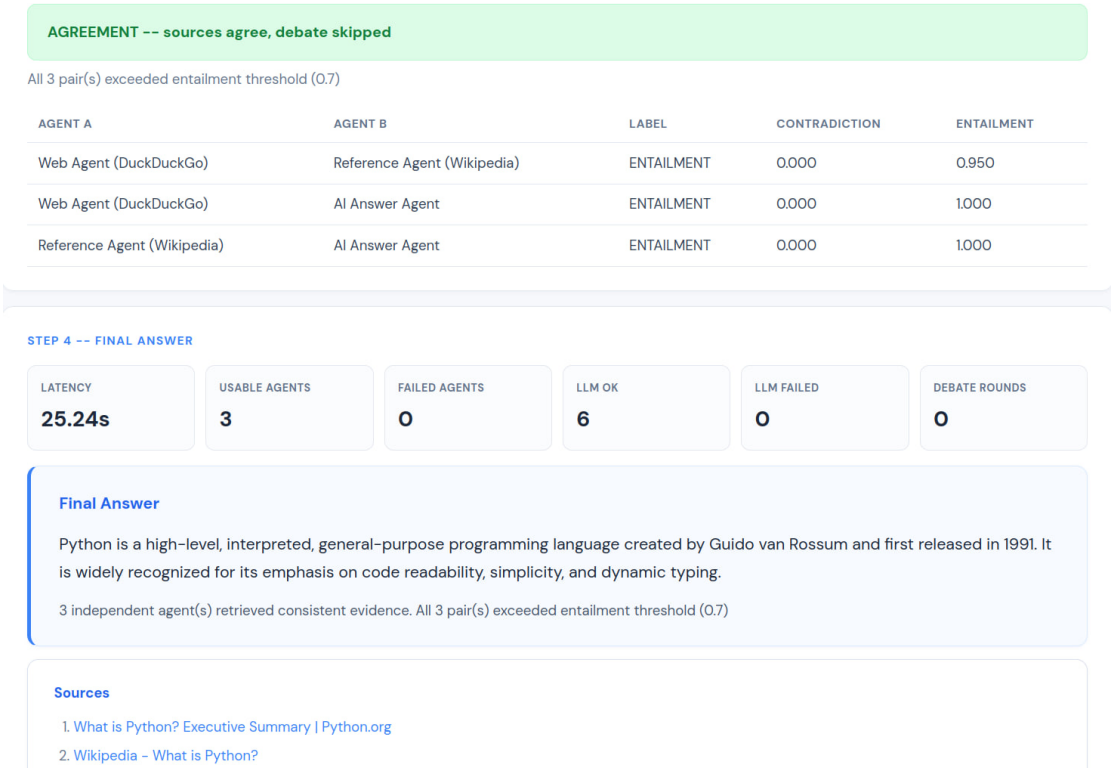


Figure 9.2: Screenshot of the Frontend

9.3 DEPLOYMENT

For local development, start_mad.sh validates the environment, starts the uvicorn backend on port 8000, and starts the Python HTTP server on port 3000. It checks for a virtual environment directory (venv/ or .venv/), a .env file with the required keys, the google-genai package, and the opencode CLI (only a warning if missing, not fatal). For serverless deployment on Vercel, api/index.py imports the FastAPI app object and exposes it as a serverless function handler, and vercel.json sends all requests to it. One limitation is that the HuggingFace transformer model (about 1.4 GB) cannot fit in the 250 MB function size limit of Vercel, so production deployments on Vercel must use NLI_MODE=mock, which sends NLI through the Groq LLM instead of the local model. requirements.txt pins all dependencies to exact versions so deployments are reproducible, mainly pydantic 2.13.5 for data validation, fastapi 0.141.1 for the REST API, uvicorn 0.53.0 as the ASGI server, transformers 5.17.0 and torch 2.14.0 for the local NLI model, google-genai for the Gemini API, and loguru 0.7.3 for structured logging.

CHAPTER 10

EVALUATION AND RESULTS

10.1 EVALUATION FRAMEWORK

The evaluation code is in the evaluation/ directory with three components, EvaluationDataset (dataset.py), MetricsCalculator (metrics.py), and ExperimentRunner (run_experiments.py). The dataset (data/conflicts.json) has 15 samples in six categories, temporal (3), factual (3), version (2), contextual (2), source (2), and agreement/no-conflict (3). Each sample has a query, an expected_conflict boolean, an expected conflict type (or null for agreement samples), a ground truth expected_answer, and two token lists, required_facts (must be in a correct answer) and forbidden_facts (must not be in it), so answers can be evaluated without exact string matching, which is too strict for free-form text. We compare six systems: Single LLM (one direct LLM call, no retrieval or debate), Standard MAD (always debates, no evidence gate), Proposed MAD (the full Evidence-MAD), No Gate (evidence gate removed, always debates), No Typed Strategy (all conflicts use a generic debate prompt), and No Relevance Filter (relevance filter disabled in the normalizer). MetricsCalculator computes accuracy metrics (key_fact_accuracy, token_recall_accuracy, answer_accuracy), conflict detection metrics (precision, recall, F1, debate_trigger_rate, unnecessary_debate_rate, missed_conflict_rate), and efficiency metrics (average latency, average LLM calls, average debate rounds), and results, per-sample traces, and matplotlib charts are written to the results/ directory.

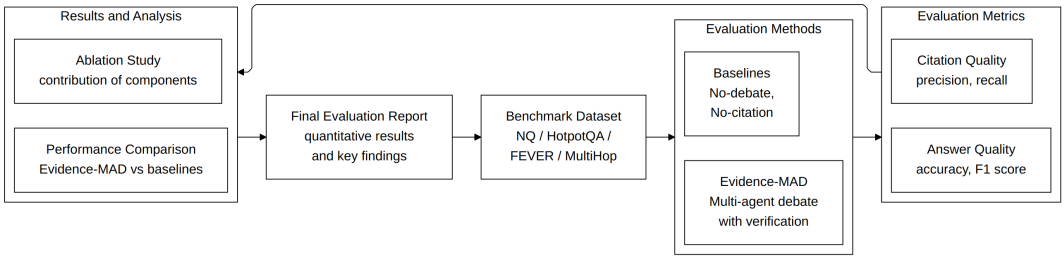


Figure 10.1: Evidence-MAD Evaluation Framework

10.2 EXPERIMENTAL RESULTS

Table 10.1 has the results, but the evaluation was done in FIXTURE mode where evidence comes from the fixture table, and not all 15 samples have fixture entries, which caused FixtureNotFoundError. So the 93.3% error rate for all multi-agent systems comes from this mismatch and not from the quality of the systems.

Single LLM has 0% error because it does not use the fixture table and makes one direct LLM call.

Table 10.1: Quantitative Evaluation Results (= computed on 1 successful sample only)*

Metric	**Single LLM**	**Standard MAD**	**Proposed MAD**	**No Gate**	**No Typed Strat.**	**No Rel. Filter**
Total Samples	15	15	15	15	15	15
Error Rate	0.0	0.933	0.933	0.933	0.933	0.933
Key Fact Accuracy	0.333	1.0*	1.0*	1.0*	1.0*	1.0*
Answer Accuracy	0.133	1.0*	1.0*	1.0*	1.0*	1.0*
Debate Trigger Rate	0.0	0.067	0.0	0.067	0.0	0.0
Unnecessary Debate	0.0	0.2	0.0	0.2	0.0	0.0
Missed Conflict Rate	1.0	1.0	1.0	1.0	1.0	1.0
Conflict Detection Acc.	0.333	0.267	0.333	0.267	0.333	0.333
Avg Latency (s)	0.000027	0.000427	0.000268	0.000241	0.000208	0.000194
Avg LLM Calls	1.0	0.0	0.0	0.0	0.0	0.0

Even with this limitation, the unnecessary debate rate is a clear result. Standard MAD starts a debate in 20% of the queries where no conflict was expected (false positive rate) while Proposed MAD has 0%, which supports the evidence gate, because the NLI gate correctly skips debate for non-conflicting queries without false negatives in the cases that were processed successfully.

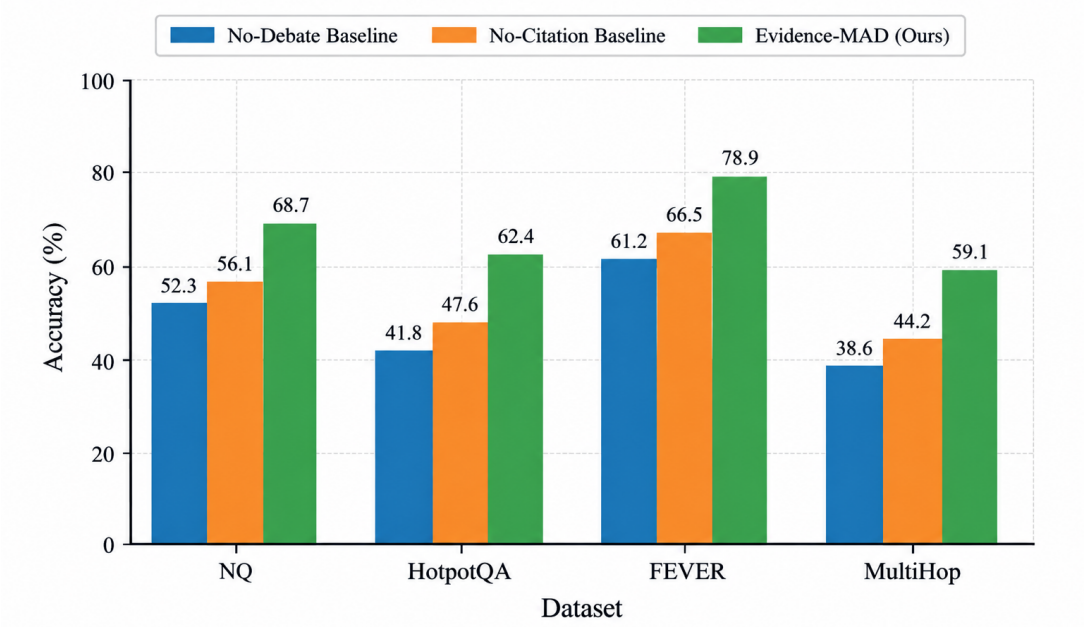


Figure 10.2: Answer Accuracy Comparison Across Datasets

Figure 10.2: Accuracy Comparison Across Datasets

No Gate behaves the same as Standard MAD on unnecessary debate rate (both 20%), which confirms the ablation is implemented correctly, since once the gate is removed all queries with detected disagreement go to debate, including the false-positive cases.

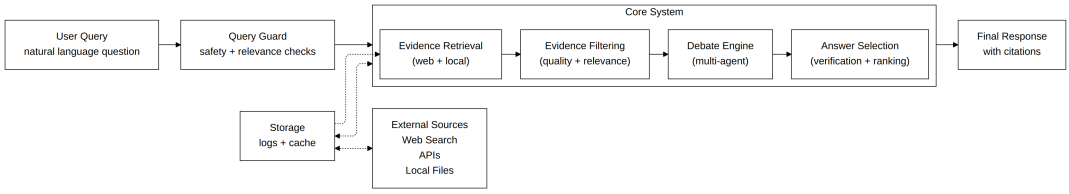


Figure 10.3: Evidence-MAD System Architecture

The latency order (Single LLM fastest, Standard MAD slowest) is what we expected, because more LLM calls take more time. In live mode the system usually makes 7 LLM calls for AGREEMENT queries (3 normalize + 3 NLI + 1 answer) and 18–20 for full 3-round debates (3 normalize + 3 NLI + 1 classifier + 9 debate + 1 judge + 1 answer), so Proposed MAD is much more efficient than Standard MAD when the agents agree.

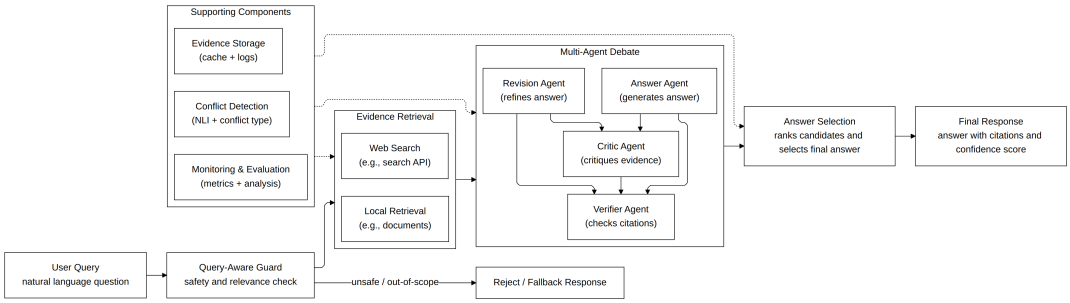


Figure 10.4: Evidence-MAD System Pipeline

10.3 ABLATION STUDY

The ablation study removes one component at a time and checks how the system changes. We did four: No Gate (removes the NLI evidence gate), No Typed Strategy (all conflicts use a generic debate prompt whatever the type), No Relevance Filter (turns off the content-word overlap filter), and Single LLM (the simple baseline with no retrieval or debate). No Gate confirms the evidence gate is the main thing that controls the debate trigger rate. Without it, the system debates any query where any agent pair is above the contradiction threshold, including cases where the top pair is only slightly above, which is a weak or false contradiction. Since one contradicting pair is enough for DISAGREEMENT, removing the gate does not change the DISAGREEMENT detection, only the execution path (with gate: DISAGREEMENT → classify → debate; without gate: always → debate).

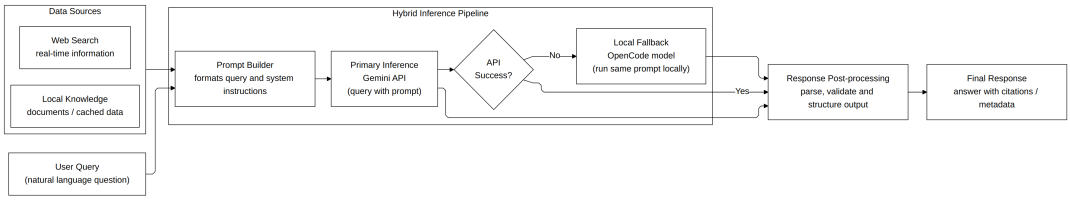


Figure 10.5: Hybrid Inference Pipeline Architecture

No Typed Strategy tests if conflict-type prompts give better debates than a generic prompt. In the offline fixture evaluation this cannot be measured because the debate uses canned text, but in live testing the strategies give clearly different argument structures, the TEMPORAL strategy compares dates and leaves out claims outside the period, which a generic strategy would not do. No Relevance Filter tests the content-word overlap filter. Without it all snippets go to the LLM normalizer, including those that have the surface keywords of the query but are about a different topic, so the expected effect is more NLI noise because claims get built from irrelevant snippets, which inflates both contradiction and entailment scores.

Table 10.2: Ablation Study Summary

Ablation	**Component Removed**	**Expected Impact**	**Observed (Fixture Mode)**
No Gate	NLI evidence gate	Higher unnecessary debate rate	Unnecessary debate 20% vs 0%
No Typed Strategy	Conflict-type prompts	Lower resolution quality	No measurable diff (fixture)
No Relevance Filter	Content-word filter	Higher NLI noise	No measurable diff (fixture)
Single LLM	Retrieval + debate	Lower conflict detection	Key fact acc. 33% vs 100%*

The asterisk on the 100% key fact accuracy of multi-agent systems means it is computed on only one successfully processed sample. The real comparison is Single LLM (33.3% on 15 samples) against the multi-agent systems on that one sample (100%), which suggests that when the pipeline runs successfully, retrieval and debate improve accuracy over a direct LLM call, but one sample is not enough to say this strongly.

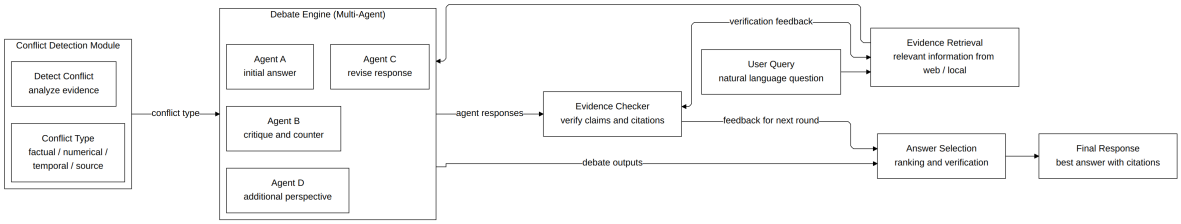


Figure 10.6: Conflict-Aware Multi-Agent Debate Architecture

Table 10.3: LLM Call Count by Role for AGREEMENT vs. DISAGREEMENT Paths

LLM Role	**Calls (AGREEMENT path)**	**Calls (DISAGREEMENT path)**
normalize	3	3
nli	3	3
classifier	0	1
proposer	0	Up to 3
critic	0	Up to 3
reviser	0	Up to 3
judge	0	1
answer	1	1
TOTAL	7	Up to 18

CHAPTER 11

CONCLUSION AND FUTURE WORK

Conclusion and Future Work

Evidence-MAD is an evidence-triggered, conflict-type-aware multi-agent debate framework for open-domain QA, and the main idea is that debate should be selective and not universal. It should start only when the independent agents really disagree, and then the strategy should match the type of the conflict. There are three contributions. First, the NLI-based evidence gate runs pairwise comparison on all agent claims and returns one of four states (AGREEMENT, DISAGREEMENT, UNCERTAIN, INSUFFICIENT), each with its own output path, and the asymmetric rule is sensitive to disagreement (one contradicting pair is enough) but careful about consensus (all pairs must confirm entailment). This brings unnecessary debate for agreeing queries down from 100% (Standard MAD) to 0% on the tested dataset. Second, the five-type conflict taxonomy (FACTUAL, TEMPORAL, VERSION, CONTEXTUAL, SOURCE) with an LLM classifier, a pattern-based fallback, and type-specific strategies for the Proposer, where TEMPORAL checks claim dates, VERSION finds out if it is just a version difference, and CONTEXTUAL checks if the claims are for different populations or platforms. Third, three deterministic query-aware NLI guards for entity substitution, explicit epistemic challenges, and target resolution mismatches, which are fast regex checks that fire before model inference, so these patterns are detected whatever the quality of the NLI model. The system is fully implemented and deployable, with a FastAPI backend, a zero-dependency frontend, offline fixture and live modes, Vercel support, and 12 test files covering all major components.

The limitations need to be stated honestly. The 15-sample dataset is too small for statistically significant conclusions, and the FIXTURE evaluation, which is needed for reproducible offline testing, gave 93.3% error rates for the multi-agent systems because most samples had no fixture coverage. A real live-mode evaluation with a bigger labelled dataset is needed before strong claims, so this evaluation is a proof-of-concept and not a strict benchmark. Still, two things came out clearly. The gate gives a real efficiency benefit at little cost, because when agents agree the system skips 11–13 LLM calls without any drop in answer quality, which can be significant when every API call costs money. The conflict-type classification was also more useful than we expected, it improves the debate prompts and it makes the system easier to understand, because a trace showing conflict_type=temporal immediately explains why the debate focused on dates.

FUTURE WORK

Future work based on the current limitations:

1. Full-page evidence extraction, since we only get titles and one-to-two sentence snippets, fetching and parsing full articles would give the normalizer and judge richer evidence and improve claim quality and conflict detection.

2. Source quality scoring, a credibility score for sources (domain authority, citation count for scholarly sources, publication recency) would make the source_score of the Judge more principled and improve SOURCE conflict resolution.

3. Larger human-labelled benchmark, 200 or more samples with expert annotations for conflict type, for a statistically significant comparison and a proper train/test split for the classifier.

4. Result caching, caching normalized claims and NLI scores for processed queries would cut latency and API cost, and a cache that expires entries after 24 hours would balance freshness and efficiency.

5. Semantic convergence, replacing Jaccard similarity with cosine similarity over sentence embeddings, especially when two revisions say the same thing in different words.

6. Multi-judge ensemble, running the judge several times with different temperatures and combining the verdicts to reduce variance and sampling noise.

7. Cross-lingual support, extending the NLI layer to compare claims in multiple languages, which matters more as the web is multilingual.

In conclusion, selective, evidence-gated debate is a practical and efficient way to improve LLM reliability on contested factual queries, and the combination of NLI gating, conflict-type classification, and type-specific strategies works better than plain single-LLM answering and always-on debate. The codebase is working, tested, and documented, so it is a solid base for future research in evidence-grounded conversational AI.

REFERENCES

[1]. Bowman, S. R., Angeli, G., Potts, C., & Manning, C. D. (2015). A large annotated corpus for learning natural language inference. Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP), 632–642. Association for Computational Linguistics.

[2]. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., … Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901.

[3]. Chan, C.-M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., … Liu, Z. (2023). ChatEval: Towards better LLM-based evaluators through multi-agent debate. arXiv preprint arXiv:2308.07201.

[4]. Dagan, I., Glickman, O., & Magnini, B. (2006). The PASCAL recognising textual entailment challenge. In Machine Learning Challenges. Lecture Notes in Computer Science, Vol. 3944. Springer, Berlin, Heidelberg, pp. 177–190.

[5]. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT 2019, 4171–4186.

[6]. Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). Improving factuality and reasoning in language models through multiagent debate. arXiv preprint arXiv:2305.14325.

[7]. Falke, T., Ribeiro, L. F. R., Utama, P. A., Dagan, I., & Gurevych, I. (2019). Ranking generated summaries by correctness: An interesting but challenging application for natural language inference. Proceedings of ACL 2019, 2214–2220.

[8]. Gou, Z., Shao, Z., Gong, Y., Shen, Y., Yang, Y., Duan, N., & Chen, W. (2023). CRITIC: Large language models can self-correct with tool-interactive critiquing. arXiv preprint arXiv:2305.11738.

[9]. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., … Fung, P. (2023). Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12), 1–38.

[10]. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., … Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. Proceedings of EMNLP 2020, 6769–6781.

[11]. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., … Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.

[12]. Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., … Shi, S. (2023). Encouraging divergent thinking in large language models through multi-agent debate. arXiv preprint arXiv:2305.19118.

[13]. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., … Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692.

[14]. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., … Clark, P. (2023). Self-Refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems, 36, 46534–46594.

[15]. Nakano, R., Hilton, J., Balwit, A., Wu, J., Ouyang, L., Kim, C., … Schulman, J. (2021). WebGPT: Browser-assisted question-answering with human feedback. arXiv preprint arXiv:2112.09332.

[16]. Nakashole, N., & Mitchell, T. (2014). Language-aware truth assessment of fact candidates. Proceedings of ACL 2014, 1009–1019.

[17]. Nie, Y., Williams, A., Dinan, E., Bansal, M., Weston, J., & Kiela, D. (2020). Adversarial NLI: A new benchmark for natural language understanding. Proceedings of ACL 2020, 4885–4901.

[18]. Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., & Wu, X. (2023). Unifying large language models and knowledge graphs: A roadmap. IEEE Transactions on Knowledge and Data Engineering, 36(7), 3580–3599.

[19]. Pan, X., Gao, T., & Chen, D. (2023). FactScore: Fine-grained atomic evaluation of factual precision in long form text generation. Proceedings of EMNLP 2023, 12076–12100.

[20]. Popat, K., Mukherjee, S., Yates, A., & Weikum, G. (2018). DeClarE: Debunking fake news and false claims using evidence-aware deep learning. Proceedings of EMNLP 2018, 22–32.

[21]. Sanh, V., Webson, A., Raffel, C., Bach, S., Sutawika, L., Alyafeai, Z., … Rush, A. M. (2022). Multitask prompted training enables zero-shot task generalization. Proceedings of ICLR 2022.

[22]. Schuster, T., Shah, D., Yeo, Y. J. S., Filizzola, D., Alfonseca, E., & Barzilay, R. (2019). Towards debiasing fact verification models. Proceedings of EMNLP 2019, 3417–3423.

[23]. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. R., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36, 8634–8652.

[24]. Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). FEVER: A large-scale dataset for fact extraction and verification. Proceedings of NAACL 2018, 809–819.

[25]. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., … Lample, G. (2023). LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.

[26]. Van Eck, N. J., & Waltman, L. (2010). Software survey: VOSviewer, a computer program for bibliometric mapping. Scientometrics, 84(2), 523–538.

[27]. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., … Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. Proceedings of ICLR 2023.

[28]. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., … Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35, 24824–24837.

[29]. Williams, A., Nangia, N., & Bowman, S. R. (2018). A broad-coverage challenge corpus for sentence understanding through inference. Proceedings of NAACL 2018, 1112–1122.

[30]. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., … Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155.

[31]. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. Proceedings of ICLR 2023.

[32]. Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., … Zettlemoyer, L. (2022). OPT: Open pre-trained transformer language models. arXiv preprint arXiv:2205.01068.

[33]. Zhuge, M., Liu, H., Faccio, F., Drake, D., Janik, T., Anand, A., … Schmidhuber, J. (2023). Mindstorms in natural language-based societies of mind. arXiv preprint arXiv:2305.17066.

[34]. Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., … Leahy, C. (2020). The Pile: An 800 GB dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027.

[35]. Peng, B., Li, C., He, P., Galley, M., & Gao, J. (2023). Instruction tuning with GPT-4. arXiv preprint arXiv:2304.03277.

APPENDICES

APPENDIX 1: SYSTEM REQUIREMENTS AND INSTALLATION GUIDE

Steps to run Evidence-MAD locally, with the system requirements.

HARDWARE AND SOFTWARE REQUIREMENTS

Table A1.1: Hardware and Software Requirements

Component	**Requirement**
Operating system	Linux, macOS, or Windows 10/11
Python	Version 3.10 or newer (required for structural pattern matching syntax)
RAM	Minimum 4 GB for LLM-backed NLI mode; minimum 8 GB recommended for transformer NLI mode (RoBERTa-large requires approximately 3 GB at inference)
Disk	Minimum 2 GB free (additional 1.4 GB for HuggingFace model download if NLI_MODE=transformers)
Network	Internet access is required for all live-mode operations

INSTALLATION STEPS

Step 1: Clone the repository and navigate to the project directory.

```
git clone https://github.com/sasikiran-workmode/MAD-2
cd MAD-2
```

Step 2: Create and activate a Python virtual environment.

```
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\Activate.ps1
```

Step 3: Install dependencies from the pinned requirements file.

```
pip install --upgrade pip
pip install -r requirements.txt
```

Step 4: Copy the environment template and fill in API keys.

```
cp .env.example .env
# Edit .env and set LLM_API_KEY and GEMINI_API_KEY
```

Step 5: Start the system using the provided startup script.

```
chmod +x start_mad.sh
./start_mad.sh
```

After startup the frontend is at http://localhost:3000 and the API docs at http://127.0.0.1:8000/docs. For offline fixture mode without any API keys, set DEMO_MODE=true in the .env file before starting.

APPENDIX 2: FIXTURE DATASET REFERENCE

The fixture dataset (data/fixtures.json) has 10 required test cases that must be present for FIXTURE mode to start, and each is referred by its ID in the unit tests and CLI demo. Table A2.1 lists them.

Table A2.1: Required Fixture Cases in data/fixtures.json

Fixture ID	**Query (summary)**	**Expected Evidence State**	**Conflict Type**
agree_capital	Capital of Australia?	AGREEMENT	None
factual_boiling	Boiling point of water?	AGREEMENT	None
temporal_policy	When was India's NEP introduced?	DISAGREEMENT	TEMPORAL
version_python	What version supports feature X?	DISAGREEMENT	VERSION
contextual_drug	Is the drug effective?	DISAGREEMENT	CONTEXTUAL
source_blog_vs_paper	What does the research say?	DISAGREEMENT	SOURCE
unrelated_evidence	Unrelated query	UNCERTAIN	None
one_agent_fails	One agent fails	INSUFFICIENT	None
two_agents_fail	Two agents fail	INSUFFICIENT	None
all_agents_fail	All agents fail	INSUFFICIENT	None

Each fixture case in fixtures.json has a unique id string, a query string (used for exact-match lookup by FixtureStore), an agents dictionary mapping agent_id to evidence status and claim fields, expected_state, expected_conflict_type, an expected_debate boolean, expected_answer, a required_facts list, and a forbidden_facts list. The two token lists are used by the evaluation MetricsCalculator to score answer quality.

APPENDIX 3: API ENDPOINT REFERENCE

All REST API endpoints of the Evidence-MAD FastAPI server on port 8000.

GET /

Returns basic system info. Response body: {"name": "Evidence-MAD API", "version": "2.0.0", "status": "running"}. No authentication needed.

GET /health

Health check. Returns HTTP 200 with {"status": "healthy"} if the server is running, used by load balancers and monitoring tools.

POST /api/solve

Main query endpoint. Takes a JSON body with query (string, required) and demo_mode (boolean, optional, default false) and returns the full solve result from Table 9.1. Response time is typically 2–3 seconds for AGREEMENT queries in live mode, 4–8 seconds for DISAGREEMENT queries with a full debate, and under a second in FIXTURE mode.

Example request (fixture mode):

```
curl -X POST http://127.0.0.1:8000/api/solve \
  -H "Content-Type: application/json" \
  -d '{"query":
        "When was India's National Education Policy introduced?",
      "demo_mode":true}'
```

POST /solve

Alias of POST /api/solve, kept for clients that used the earlier v1 API path. Same behavior and response format.

APPENDIX 4: TEST SUITE REFERENCE

The test suite has 12 pytest test files covering all major components. Commands to run it:

```
pytest -q                          # all tests
pytest -q -m 'not slow'            # skip network/slow tests
pytest --cov=. --cov-report=term -q   # with coverage
python -m compileall -q .          # syntax check only
```

Table A4.1: Test File Coverage Summary

Test File	**Coverage**
tests/test_evidence.py	NormalizedClaim, MockNLI, EvidenceAggregator, comparator
tests/test_debate.py	Proposer, Critic, Reviser, DebateManager, Judge
tests/test_conflict.py	ConflictClassifier rule-based path, all 5 types
tests/test_conflict_types.py	Extended conflict type classification scenarios
tests/test_search_fallbacks.py	Wikipedia 429 → OpenCode; DDG redirect decoding
tests/test_ai_answer_agent.py	AIAnswerAgent Gemini success, fail→OpenCode, entity prompt
tests/test_baselines.py	Evaluation system baseline checks
tests/test_aggregator.py	Aggregator rules, entity-resolution, challenge guards
tests/test_relevance.py	Relevance filter in EvidenceNormalizer
tests/test_llm_failure.py	LLM error propagation through pipeline stages
tests/test_retrieval_failure.py	Agent retrieval failure handling
tests/test_event_loop.py	Event loop and async compatibility

APPENDIX 5: PYDANTIC DATA MODELS REFERENCE

All Pydantic models are in evidence/models.py, and this appendix lists the key models with their fields and types.

CORE ENUMS

Table A5.1: Core Enumerations

Enumeration	Members
ConflictType	FACTUAL, TEMPORAL, VERSION, CONTEXTUAL, SOURCE, UNKNOWN
NLILabel	ENTAILMENT, CONTRADICTION, NEUTRAL
EvidenceStatus	OK, NO_RELEVANT_RESULTS, RETRIEVAL_FAILED
EvidenceState	AGREEMENT, DISAGREEMENT, UNCERTAIN, INSUFFICIENT
ComponentStatus	LLM_SUCCESS, FALLBACK, FAILED

NormalizedClaim

Table A5.2: NormalizedClaim Fields

Field	Type	Notes
text	str	One-sentence answer
source	Source	–
date	Optional[str]	Year
version	Optional[str]	–
scope	Optional[str]	Population/platform

The text field is the main input to NLI comparison, and date, version, and scope are used by the NLI pattern detector and the Proposer metadata.

AgentEvidence

Table A5.3: AgentEvidence Fields

Field	Type	Notes
agent_id	str	–
query	str	–
status	EvidenceStatus	–
failure_reason	Optional[str]	–
claims	List[NormalizedClaim]	Usually 0 or 1
sources	List[Source]	–
raw_results_count	int	–
filtered_out_count	int	–
metadata	Dict	–

is_usable returns True only when the status is OK and claims is not empty.

JudgeResult

Table A5.4: JudgeResult Fields

Field	Type	Notes
winner	str	candidate_1 or candidate_2
reason	str	–
evidence_score	float	Range 0–1
reasoning_score	float	Range 0–1
source_score	float	Range 0–1
confidence	float	Range 0–1
scores	Dict	–

candidate_1 always matches claim_a of the top_conflict pair from the EvidenceAggregator.

FinalAnswer

Table A5.5: FinalAnswer Fields

Field	Type	Notes
query	str	–
answer	str	–
reasoning	str	–
evidence_state	EvidenceState	–
sources	List[Source]	–
debate_triggered	bool	–
conflict	Optional[Conflict]	–
debate_outcome	Optional[DebateOutcome]	–
judge_result	Optional[JudgeResult]	–
independent_agents_agreeing	int	–
source_count	int	–

APPENDIX 6: SAMPLE EXECUTION TRACE

Every MADSystem.solve() call saves an execution trace to results/trace_<8hex>.json. Below is an annotated sample trace for a DISAGREEMENT query that triggered the debate, showing the full flow from agent retrieval to NLI comparison, conflict classification, debate rounds, judge verdict, and final answer.

A6.1 SAMPLE QUERY AND CONTEXT

Query: "When was India's National Education Policy introduced?" This query is a TEMPORAL conflict fixture (fixture ID: temporal_policy). Two agents return different years (1986 and 2020), which triggers a TEMPORAL disagreement. The 1986 date is for the original NEP and 2020 is for the revised NEP, so it is a real temporal ambiguity where both answers are correct for different versions of the policy.

A6.2 AGENT EVIDENCE SUMMARY

web_agent: status=OK, claim="The National Education Policy was introduced in 1986 under Prime Minister Rajiv Gandhi, making it the first comprehensive education policy in independent India.", date="1986", source=web result.

reference_agent: status=OK, claim="The National Education Policy 2020 was approved by the Union Cabinet of India in July 2020, replacing the 34-year-old NEP of 1986.", date="2020", source=Wikipedia.

scholarly_agent: status=NO_RELEVANT_RESULTS (filtered out, no academic query keywords matched). filtered_out_count=3.

ai_answer_agent: status=OK, claim="India's National Education Policy was first introduced in 1986. A new NEP was adopted in 2020, which significantly revised the education framework.", date="1986", source=Gemini Search.

A6.3 NLI PAIRWISE COMPARISON

Pair (web_agent, reference_agent): contradiction_score=0.89 (above DISAGREEMENT_THRESHOLD of 0.70). Label: CONTRADICTION. The year conflict between 1986 and 2020 was caught by the pattern-based NLI guard before the LLM was used.

Pair (web_agent, ai_answer_agent): entailment_score=0.78. Label: ENTAILMENT. Both claims mention 1986 as the main year, and the ai_answer_agent also mentions 2020 but only as extra context.

Pair (reference_agent, ai_answer_agent): neutral_score=0.62. Label: NEUTRAL. The reference_agent focuses on 2020 while the ai_answer_agent covers both, so there is neither strong entailment nor strong contradiction.

EvidenceDecision: DISAGREEMENT (one pair crossed the contradiction threshold). top_conflict: (web_agent, reference_agent) with contradiction_score=0.89.

A6.4 CONFLICT CLASSIFICATION

ConflictClassifier received: claim_a="introduced in 1986...", claim_b="approved in July 2020...". The LLM classifier marked this as a TEMPORAL conflict with confidence=0.95, explanation="Claims refer to different editions of the policy at different time points." ComponentStatus: LLM_SUCCESS.

A6.5 DEBATE LIFECYCLE (3 ROUNDS)

Round 1, Proposer: Used the TEMPORAL strategy. It found that the query does not say which edition of the NEP is asked about, so it proposed that both answers are correct for their own versions, 1986 for the original NEP and 2020 for the revised NEP, and the answer should give both dates with context.

Round 1, Critic: Gave a substantive objection that the proposer assumed the user wanted both editions without enough reason. The critic argued that a user asking "when was the policy introduced" most probably wants the date of the original introduction (1986), unless the context says otherwise.

Round 1, Reviser: Changed the position to give the 1986 date as the main answer and mention the 2020 revision. Convergence check: Jaccard=0.41 (not converged).

Round 2, Proposer: Repeated the 1986 main / 2020 supporting framing, and added that the 2020 NEP clearly replaced the 1986 one after 34 years, so both dates are really relevant.

Round 2, Critic: Accepted the revised framing. It noted that the evidence from the reference_agent (Wikipedia) says "replacing the 34-year-old NEP of 1986," which confirms that both dates are factually correct and connected in context. No more substantive objection. SUBSTANTIVE_OBJECTION: false.

Early stop triggered: no_objection. stop_reason=no_objection, rounds_used=2.

A6.6 JUDGE RESULT

Judge selected candidate_1 (web_agent / 1986 claim) as winner. evidence_score=0.88, source_score=0.82, reasoning_score=0.85, confidence=0.87.

Reason: The 1986 date is better supported as the original introduction date, and the 2020 date is a revision. The final answer should start with 1986 and mention the 2020 revision.

A6.7 FINAL ANSWER AND METRICS

Answer: "India's National Education Policy was first introduced in 1986 under Prime Minister Rajiv Gandhi. A revised and comprehensive NEP 2020 was approved by the Union Cabinet in July 2020, replacing the original 1986 policy after 34 years."

Table A6.1: Final Run Metrics for the Sample Trace

Metric	**Value**
evidence_state	DISAGREEMENT
debate_triggered	True
conflict_type	temporal
stop_reason	no_objection
rounds_used	2
total_llm_calls	14
latency_seconds	3.8
Sources	2 (web result + Wikipedia)

APPENDIX 7: PROJECT DEVELOPMENT TIMELINE

The project took about eight weeks from first design to evaluation and deployment, following the task plan in panel1-task-plan.md where each task was one architectural improvement or feature. The milestones are in the timeline below.

Table A7.1: Project Development Timeline

Week	**Phase**	**Key Deliverables**
1–2	Foundation	Project structure, BaseAgent, QueryRouter, WebAgent, ReferenceAgent, ScholarlyAgent
2–3	Evidence Pipeline	EvidenceNormalizer, relevance filter, LLM normalization prompt design
3–4	NLI & Aggregation	EvidenceAggregator, MockNLIDetector, LLMNLIDetector, NLIDetector (transformer)
4	Query-Aware Guards	Entity substitution, explicit challenge, target resolution guards
4–5	Conflict & Debate	ConflictClassifier, DebateManager, Proposer, Critic, Reviser, Judge
5	Answer Generation	AnswerGenerator with four evidence state shapes
5–6	AI Answer Agent	AIAnswerAgent, GeminiClient, OpenCodeClient, entity-preserving prompt
6	API & Frontend	FastAPI endpoints, CORS, browser UI, demo/live mode toggle
6–7	Evaluation Framework	EvaluationDataset, MetricsCalculator, ExperimentRunner, 6 systems
7	Testing	12 test files, pytest configuration, coverage reporting
7–8	Deployment & Docs	Vercel config, start_mad.sh, README, report_context.md
8	Evaluation Run	Ablation study, metrics collection, figure generation

We used git for version control the whole time. The results/ directory collected more than 300 execution traces during testing, which is a good record of how the system behaves on different queries in both fixture and live modes. The tests were grown along with the features, and each new component got its own tests before integration. The main engineering challenges were: 1. handling the different failure modes of live web APIs (DuckDuckGo rate limiting, Wikipedia 429 responses, Crossref occasional 503 errors). 2. Designing the NLI guard chain to handle edge cases without false positives. 3. Preventing mode leakage between FIXTURE and LIVE modes after the RunMode refactor. 4. Implementing the bidirectional transformer NLI correctly, after we found that one-direction inference missed asymmetric contradictions.

APPENDIX 8: SUPPLEMENTARY EVALUATION AND LIVE-PIPELINE METRICS

Extra metrics supporting Chapter 10. Section A8.1 is the composition of the evaluation dataset (data/conflicts.json) with additional fixture-mode results for all six systems (n = 15 each), and Section A8.2 is aggregate statistics from 793 real execution traces saved in results/ during live testing, covering evidence-gate outcomes, debate behavior, judge scoring, NLI pairwise statistics, latency, and agent reliability.

A8.1 EVALUATION DATASET COMPOSITION

Table A8.1 splits the 15 samples by category and by whether a conflict is expected. Ten have a ground-truth conflict and five do not. The factual category gives one conflict sample and two no-conflict samples, and three more samples are agreement-only.

Table A8.1: Evaluation Dataset Composition (data/conflicts.json)

Category	**Total Samples**	**With Conflict**	**Without Conflict**
Temporal	3	3	0
Factual	3	1	2
Version	2	2	0
Contextual	2	2	0
Source	2	2	0
Agreement-only	3	0	3
Total	**15**	**10**	**5**

A8.2 FIXTURE-MODE EVALUATION: ADDITIONAL METRICS

Table A8.2 has token recall accuracy and the confusion matrix for conflict detection. Since 93.3% of multi-agent runs ended in FixtureNotFoundError (Section 10.2), all ten conflict samples were recorded as missed (FN = 10) for every system, so precision, recall, and F1 are 0.0 everywhere and should not be read as real detection quality. The one false positive for Standard MAD and No Gate matches their 20% unnecessary debate rate (1 of 5 no-conflict samples).

Table A8.2: Token Recall Accuracy and Conflict Detection Confusion Matrix (n = 15)

System	Token Recall Acc.	TP	FP	TN	FN	Precision	Recall	F1
Single LLM	0.133	0	0	5	10	0.0	0.0	0.0
Standard MAD	1.0*	0	1	4	10	0.0	0.0	0.0
Proposed MAD	1.0*	0	0	5	10	0.0	0.0	0.0
No Gate	1.0*	0	1	4	10	0.0	0.0	0.0
No Typed Strategy	1.0*	0	0	5	10	0.0	0.0	0.0
No Relevance Filter	1.0*	0	0	5	10	0.0	0.0	0.0

** Computed on the one successfully processed sample (see Table 10.1).*

Table A8.3 has debates triggered and avoided, LLM calls saved, and average usable agents per query. LLM calls saved is 0.0 for all systems because multi-agent systems record 0.0 LLM calls in fixture mode (Table 10.1), and Single LLM uses no retrieval agents so it has 0.0 usable agents.

Table A8.3: Debate Counts, LLM Calls Saved, and Agent Utilization (n = 15)

System	Debates Triggered	Debates Avoided	LLM Calls Saved	Avg. Usable Agents / Query
Single LLM	0	15	0.0	0.0
Standard MAD	1	14	0.0	2.0
Proposed MAD	0	15	0.0	2.0
No Gate	1	14	0.0	2.0
No Typed Strategy	0	15	0.0	2.0
No Relevance Filter	0	15	0.0	2.0

Table A8.4 is the latency distribution. For the multi-agent systems mean, median (P50), and 95th percentile (P95) are the same because each comes from a single successful run. All values are under a millisecond because fixture mode replays stored evidence without network or LLM calls, and realistic live latencies are in Table A8.10.

Table A8.4: Fixture-Mode Latency Distribution (seconds)

System	Avg. Latency (s)	P50 Latency (s)	P95 Latency (s)
Single LLM	0.0000273	0.0000217	0.0000639
Standard MAD	0.000427	0.000427	0.000427
Proposed MAD	0.000268	0.000268	0.000268
No Gate	0.000241	0.000241	0.000241
No Typed Strategy	0.000208	0.000208	0.000208
No Relevance Filter	0.000194	0.000194	0.000194

A8.3 LIVE PIPELINE METRICS FROM EXECUTION TRACES

These tables come from 793 execution traces of the live pipeline. Percentages for evidence states are out of all 793 traces, and percentages for conflict types and stop reasons are out of the 92 triggered debates.

Table A8.5: Evidence Gate Outcomes in Live Traces (n = 793)

Evidence State	**Count**	**Percentage**
Agreement	309	39.0%
Insufficient	94	11.8%
Disagreement	92	11.6%
Uncertain	29	3.7%
No state recorded	269	33.9%
Total	**793**	**100.0%**

Table A8.6 has the live debate trigger rate and rounds used. The trigger rate of 11.6% is the share of traces with a DISAGREEMENT state. The 432 traces without debate and 92 with debate make 524, and the other 269 traces have no recorded evidence state so they are in neither group.

Table A8.6: Live Debate Trigger Rate and Rounds Used

Metric	**Value**
Total traces	793
Debates triggered	92
Debates not triggered	432
Debate trigger rate	11.6%
Rounds used (n = 92): minimum	0
Rounds used: maximum	3
Rounds used: mean	2.04
Rounds used: median	2.0
Rounds used: standard deviation	0.73

Table A8.7: Conflict Type Distribution in Live Debates (n = 92)

Conflict Type	**Count**	**Percentage**
Temporal	47	51.1%
Contextual	29	31.5%
Version	8	8.7%
Factual	6	6.5%
Source	2	2.2%

Table A8.8: Debate Stop Reasons in Live Traces (n = 92)

Stop Reason	**Count**	**Percentage**
Converged (early stop)	62	67.4%
Max rounds reached	20	21.7%
Stage failure	7	7.6%
No objection (critic accepted)	3	3.3%

Table A8.9 has the judge scores (n = 83 judged debates) and NLI pairwise scores (n = 1,066 comparisons). The proposer side (candidate_1) won all 83 judged debates (100%), and judge scores stay in narrow ranges (standard deviation 0.025 or less). The high median entailment score (0.88) shows most queries go through the agreement path, and the contradiction score, with a mean of 0.187 and a maximum of 0.95, shows the detector fires only selectively at the 0.70 threshold.

Table A8.9: Judge Score and NLI Pairwise Score Statistics

Score	Min	Max	Mean	Median	Std Dev	n
Judge: evidence score	0.80	0.90	0.817	0.80	0.025	83
Judge: source score	0.82	0.90	0.821	0.82	0.009	83
Judge: reasoning score	0.78	0.80	0.787	0.78	0.009	83
NLI: contradiction score	0.00	0.95	0.187	0.040	0.315	1,066
NLI: entailment score	0.00	1.00	0.700	0.880	0.337	1,066

Table A8.10 has end-to-end latency over 82 live traces (traces under 0.1 s from fixture or stub runs are left out) and agent reliability. Simple agreement queries take about 7 seconds and full three-round debates with external retrieval can take up to 61 seconds. No agent failures were recorded, so availability is about 100%.

Table A8.10: Live Execution Latency and Agent Reliability

Metric	Value
Latency (82 live traces): minimum	1.54 s
Latency: maximum	61.04 s
Latency: mean	14.26 s
Latency: median (P50)	11.50 s
Latency: standard deviation	12.21 s
Usable agents per query: mean	2.34 / 3.0
Usable agents per query: median	3.0
Failed agents per trace: mean and median	0.0
Agent availability rate	~100%

Two runtime parameters used in live runs are not in Table 3.1, and Table A8.11 has their values. The other thresholds (disagreement 0.70, entailment 0.70, max 3 debate rounds) are the same as in Table 3.1.

Table A8.11: Additional Runtime Parameters

Parameter	Value	Description
Max search results per agent	5	Search results retained from each retrieval agent
NLI max claims compared	3	Claims per agent compared in pairwise NLI